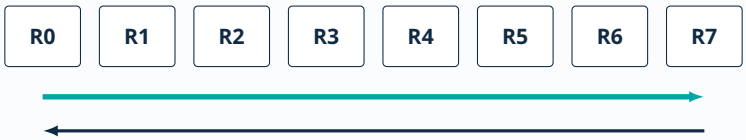
HIGH-PERFORMANCE

16-BIT ARCHITECTURE

LIM M2

Architektonická príručka procesora

Vysoko deterministická 16-bitová mikroprogramová architektúra s čistou multiplexorovou dátovou cestou a rozšíreným 24-bitovým adresovaním.



POINT-TO-POINT DATA ROUTING · ZERO BUS CONTENTION

OFFICIAL REFERENCE SPECIFICATION

Revízia 0.2 — Október 2026

Špecifikácia jadra LIM M2. Všetky práva vyhradené.

Contents

1 Rozsah dokumentu a architektonická filozofia 3
1.1 Dohodnuté označenia . . . 3
1.2 Jednotná filozofia formátu inštrukcií . . . 3

2 Architektonický prehľad procesora LIM M2 3
2.1 Konceptuálna schéma dátovej cesty (Datapath) . . . 5

3 Programovo viditeľné registre a stav procesora 6
3.1 Topológia príznakového a riadiaceho registra (FREG) . . . 6

4 Formát strojových inštrukcií 7
4.1 Univerzálna bitová šablóna strojovej inštrukcie . . . 7
4.2 Trieda I: Jednobajtové inštrukcie (1 bajt, 0 doplňujúcich oktetov) . . . 7
4.3 Trieda II: Trojoperandové aritmetické inštrukcie (2 bajty) . . . 7
4.4 Trieda III: Dvojoperandové logické inštrukcie (2 bajty) . . . 8
4.5 Trieda IV: Medziregistrový presun LRR (2 bajty) . . . 8
4.6 Trieda V: Hardvérový komparátor CMP (2 bajty) . . . 8
4.7 Trieda VI: Hardvérový cyklický posun na D-klopných obvodoch (BSL, BSR) . . . 8
4.8 Trieda VII: Operácie so zásobníkom (PUSH, POP) . . . 9
4.9 Trieda VIII: Manipulácia so špeciálnymi registrami (MSP, Opkód 26) . . . 9
4.9.1 Formát 8.1: Načítanie do príznakového registra FREG (MOD = 0, REG = 00) . . . 9
4.9.2 Formát 8.2: Bitová logika FREG (MOD = 1, REG = 00) . . . 9
4.9.3 Formát 8.3: Načítanie ukazovateľov PTREG a SREG (MOD = 0, REG = 01 / 10) . . . 10
4.9.4 Formát 8.4: Aritmetika posunu PTREG a SREG (MOD = 1, REG = 01 / 10) . . . 10
4.9.5 Formát 8.5: Načítanie 32-bitových vektorových registrov (MOD = 0, REG = 11) 10
4.10 Trieda IX: Inštrukcie vetvenia a skokov (2–3 bajty) . . . 11
4.11 Trieda X: Inštrukcie výmeny s pamäťou LMR a LRM (2–4 bajty) . . . 11

5 Prehľadná tabuľka inštrukcií jadra LIM M2 11

6 Detailná špecifikácia inštrukčného súboru 12
NOP — Hardvérová prázdna operácia a vyprázdnenie aritmetickej reťaze . . . 13
ADD — Sčítanie s automatickým reťazením prenosu . . . 15
SUB — Odčítanie s automatickým reťazením výpožičky . . . 17
MUL — Celočíselné násobenie so zachovaním hornej časti v registri ALU . . . 19
DIV — Celočíselné delenie so zachovaním zvyšku vo vnútornom registri ALU . . . 21
LMR — Načítanie dát z pamäte a externých zariadení do registrov . . . 23
LRM — Zápis dát z registrov do pamäte a externých zariadení . . . 25
LRR — Medziregistrový presun (kopírovanie z bodu B do bodu A) . . . 27

CMP — Hardvérové porovnanie operandov . . . 29
JMP — Nepodmienený skok . . . 31
JCR — Podmienený skok pri prenose . . . 33
JNZ — Podmienený skok pri nenulovom stave . . . 35
JZ — Podmienený skok pri nulovom stave . . . 37
AND — Bitový logický súčin (AND) . . . 39
OR — Bitový logický súčet (OR) . . . 41
NAND — Bitový logický zápor súčinu (NAND) . . . 43
NOR — Bitový logický zápor súčtu (NOR) . . . 45
NOT — Bitová inverzia (jednotkový doplnok) . . . 47
XOR — Bitová neverzia (XOR) . . . 49
XNOR — Bitová ekvivalencia . . . 51
INC — Inkrementácia o 1 . . . 53
DEC — Dekrementácia o 1 . . . 55
PUSH — Uloženie slova do zásobníka . . . 57
POP — Výber slova zo zásobníka . . . 59
BSL — Hardvérový posun doľava (iteratívny register) . . . 61
BSR — Hardvérový posun doprava (iteratívny register) . . . 63
MSP — Manipulácia so špeciálnymi registrami (FREG, PTREG, SREG, VEC) . . . 65
MOD — Celočíselný zvyšok po delení (modulo) . . . 67
INTR — Softvérové prerušenie podľa tabuľky vektorov (2 bajty) . . . 69
CALL — Volanie podprogramu s uložením PC na zásobník . . . 71
RET — Návrat z podprogramu s obnovením PC zo zásobníka . . . 73
HALT — Hardvérové zastavenie procesora . . . 75

7 Fyzické rozhranie a rozloženie vývodov (50-vývodové puzdro) 77
7.1 Funkčný popis signálov . . . 77

8 Podsystém prerušení a vektorové spracovanie 78
8.1 Zjednotený priestor 16 vektorov prerušení . . . 78
8.2 Hardvérový protokol potvrdzovania (RDI → IA → D0–D7) . . . 78
8.3 Uloženie kontextu v 24-bitovom zásobníku a vstup do obsluhy . . . 79
8.4 Návrat z prerušenia . . . 79

9 Jazyk symbolických inštrukcií (Assembler) a kompilátor LIM M2 (`limasm.py`) 80
9.1 Pomenovanie registrov a prípony veľkosti operandov . . . 80
9.2 Aritmetika, unárne inštrukcie a prístup do pamäte (LMR, LRM) . . . 80
9.3 Špeciálne registre (MSP) a viacnásobné dereferencovanie cez PTREG . . . 80

10 Záver 80

1 Rozsah dokumentu a architektonická filozofia

Tento dokument predstavuje oficiálnu technickú špecifikáciu a architektonický štandard mikroprocesora **LIM M2**. Architektúra bola navrhnutá s dôrazom na maximálnu spoľahlivosť, matematickú exaktnosť a vysokú taktovaciu frekvenciu.

Základným kameňom dátovej cesty LIM M2 je **úplné vylúčenie vnútorných zberníc tretieho stavu (Tri-State / Hi-Z)**. Nahradením zdieľaných kapacitných liniek aktívne budenými stromami multiplexorov (Point-to-Point MUX Routing) sa úplne zamedzuje vzniku skratových prúdov, zbernicových konfliktov (bus contention) a parazitných kapacitných oneskorení.

> **Číslovanie inštrukčného súboru**
> Inštrukčná sada LIM M2 využíva súvislý 5-bitový priestor primárnych kódov: `NOP`=0, `ADD`=1, ..., `MSP`=26. Kódy 27–31 sú vyhradené pre hardvérové rozšírenia novej generácie.

1.1 Dohodnuté označenia

Označenie	Architektonický význam
`dst`	Cieľový register pre zápis výsledku operácie
`src, srcA, srcB`	Zdrojové registre vstupných operandov
`imm`	Priama skalárna konštanta v tele inštrukcie
`addr`	Efektívna adresa pamäte (16-bitová základná alebo 24-bitová rozšírená)
`cfg`	3-bitové konfiguračné pole modifikátora operácie
`ZF, CF, OF`	Hardvérové príznaky nuly, prenosu a pretečenia v registri `FREG`

1.2 Jednotná filozofia formátu inštrukcií

Architektúra procesora LIM M2 striktne uplatňuje univerzálnu paradigmu: **najprv sa vždy uvádza cieľový príjemca (kam), a až potom vstupné operandy (odkiaľ)**. To zaručuje čistú, logickú a prehľadnú syntax:

- **Aritmetické operácie (3 operandy):** `ADD Rd, Rs1, Rs2` znamená Rd $\leftarrow$ Rs1 $+$ Rs2 (napríklad zápis `ADD R0, R1, R2` vypočíta $R0 = R1 + R2$); rovnako pre `SUB`, `MUL`, `DIV`.
- **Logické operácie (2 operandy):** `AND Rd, Rs` znamená Rd $\leftarrow$ Rd AND Rs (napríklad `AND R0, R1` vypočíta $R0 = R0 \ \& \ R1$); rovnako pre `OR`, `NAND`, `NOR`, `XOR`, `XNOR`.
- **Jednobajtové unárne operácie (1 operand):** `INC Rd` (Rd $\leftarrow$ Rd $+ 1$), `DEC Rd` (Rd $\leftarrow$ Rd $- 1$) a `NOT Rd` (Rd $\leftarrow \sim$ Rd). Pracujú s jediným cieľovým registrom Rd na mieste. Kódujú sa presne v 1 bajte (`[7:3]` opkód, `[2:0]` register Rd), druhý bajt zo zbernice pamäte sa nenačítava.
- **Medziregistrové presuny:** `LRR Rd, Rs` vykonáva priame kopírovanie z bodu B (zdroj Rs) do bodu A (príjemca Rd): Rd $\leftarrow$ Rs (napríklad `LRR R0, R1` vykoná $R0 = R1$).
- **Načítanie z pamäte (LMR):** prvý argument Rd určuje cieľový register procesora, do ktorého sa uložia načítané dáta.

2 Architektonický prehľad procesora LIM M2

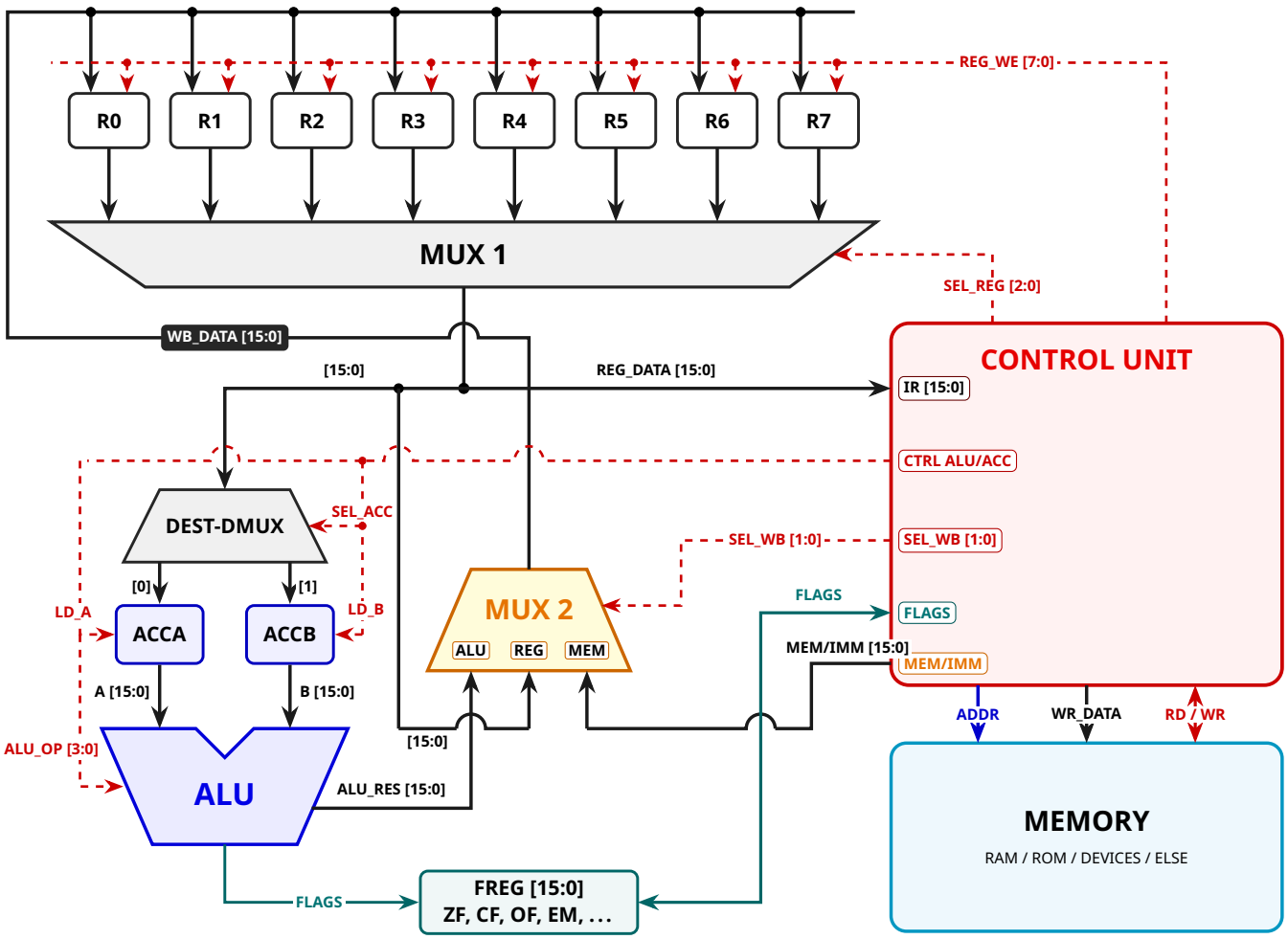
LIM M2 je všestranný, vysoko výkonný 16-bitový procesor s mikroprogramovým riadením, optimalizovaný na celočíselné výpočty. Medzi kľúčové architektonické prednosti patria:

- **Čisté multiplexorové smerovanie dát:** Vnútorná dátová cesta je kompletne zbavená tretieho

stavu. Prenos dát je riadený aktívnymi budičmi cez kaskádu selektorov **MUX 1**, **DEST-DMUX** a **MUX 2**. Architektúra je optimálne prispôsobená moderným technológiám ASIC a rýchlym FPGA.

- **Dvojitý akumulátorový uzol (Dual Accumulator Staging):** Registre ACCA a ACCB stabilne fixujú operandy pred ALU, čím izolujú prechodové javy výpočtu od hlavného registrového poľa.
- **Hardvérové reťazenie dlhej aritmetiky (ALU Auto-Chaining):** Blok ALU integruje vnútorný zvyškový register (ALU_RESIDUAL). V prípade prenosu ($+1$ pri súčte $> 2^{16} - 1$), výpožičky (-1 pri rozdiele < 0) alebo vyšších bitov násobenia/delenia sa zvyšok zachytáva hardvérovo a automaticky pripočíta či odpočíta v nasledujúcej matematickej operácii. Tým sa zabezpečuje priebežná viacbajtová aritmetika (32, 48, 64+ bitov) bez samostatných inštrukcií ADC/SBB. Vyprázdnenie reťazca sa vykonáva inštrukciou NOP.
- **Priamy premosťovací kanál (Reg-to-Reg Bypass):** Obchádzková vetva z MUX 1 priamo do MUX 2 umožňuje bleskové kopírovanie medzi registrami bez blokovania aritmetickej jednotky.
- **Natívne 32-bitové adresovanie pamäte (až 4 GB):** Natívna 32-bitová zbernica adresy (A0–A31) a špecializované 32-bitové registre PTREG (ukazovateľ údajov s autoincrementom) a SREG (zásobníkový ukazovateľ), spolu s 32 systémovými vektorovými registrami ($ir0..$ir15, $pv0..$pv15), umožňujú priamy lineárny prístup k celému 4 Gigabajtovému fyzickému pamäťovému priestoru bez segmentačných okien a prepínania bánk.
- **Vysoký frekvenčný potenciál:** Odstránenie kapacitného zaťaženia distribuovaných zberníc umožňuje jadru dosahovať základnú taktovaciu frekvenciu **200 MHz**, pričom v optimalizovanej kremíkovej implementácii škáluje na **700 MHz a viac**.

2.1 Konceptuálna schéma dátovej cesty (Datapath)



3 Programovo viditeľné registre a stav procesora

Architektonický stav procesora LIM M2 zahŕňa ortogonálnu sadu funkčných registrov:

Register	Šírka	Architektonický účel a vlastnosti
R0–R7	16 bitov	Registre všeobecného určenia (GPR). Selektor MUX 1 zabezpečuje bleskové čítanie; zápis prebieha synchronizovane cez zbernicu WB_DATA pod kontrolou REG_WE.
SREG	24 bitov	Systémový zásobníkový register (Stack Register). Hardvérová podpora volaní podprogramov a lokálnych rámcov s 24-bitovým adresovaním zásobníka (až 16 MB).
PTREG	24 bitov	Presný ukazovateľový register (Pointer Register). Zabezpečuje lineárne adresovanie pamäte až do 16 MB s voliteľným hardvérovým autoincrementom.
FREG	16 bitov	Stavový a riadiaci register (Status & Control Register). Konsoliduje aritmetické príznaky ALU a konfiguračné bity jadra.

3.1 Topológia príznakového a riadiaceho registra (FREG)

Register FREG koordinuje činnosť výpočtových a systémových uzlov procesora:

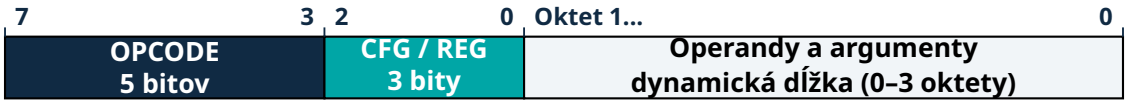
Bit	Mnemonic	Funkčný popis
0	ZF	Zero Flag: nastavený pri nulovom výsledku operácie.
1	CF	Carry Flag: príznak aritmetického prenosu alebo výpožičky.
2	OF	Overflow Flag: indikátor znamienkového pretečenia v doplnkovom kóde.
3	EM	Error Math: indikátor matematických výnimiek (delenie nulou).
4–5	CMP	Compare Result: stav hardvérového komparátora (00: reset/neporovnané/chyba, 01: menšie, 10: rovné, 11: väčšie).
6–7	CLK	Clock Mode: voľba deliča systémovej frekvencie jadra.
8	PIE	Program Interrupt Enable: globálne povolenie softvérových prerušení.
9	MIE	Master Interrupt Enable: povolenie externých hardvérových prerušení.
10	DWR	Disable Wait for Ready: nútený zbernicový cyklus bez čakania na potvrdzovací signál.
11	PRV	Privilege Mode: 0 = User Mode, 1 = Supervisor Mode (chránené 8 Zónami Privilégií $pv0..$pv15).
12	AIP	Auto-Increment PTREG: povolenie hardvérového autoincrementu ukazovateľa PTREG.
13	DBIT	Displaced Bit: záchytný register pre bit vysunutý pri posunoch.
14	SMOD	Shift Mode: voľba cyklického posunu cez DBIT (0: nulovanie, 1: priechod cez DBIT).
15	INTR	In Interrupt: príznak vykonávania prerušenia (pri 1 sú ďalšie prerušenia zablokované).

4 Formát strojových inštrukcií

Inštrukcie mikroprocesora LIM M2 majú ortogonálny, modulárny formát s deterministickým dekódovaním. Základný opkód vždy zaberá horných 5 bitov prvého oktetu, čo garantuje okamžitú hardvérovú dispečerizáciu v mikroprogramovom riadiacom bloku.

4.1 Univerzálna bitová šablóna strojovej inštrukcie

Každá strojová inštrukcia začína základnou hlavičkou pevnej veľkosti, za ktorou nasledujú dynamické operandy a rozšírenia:



Spodné 3 bity prvého oktetu (`[2:0]`) majú dvojaké deterministické určenie:

- V operáciách s registrami (aritmetika, unárne inštrukcie, logika, posuny, presuny, zásobník) bity $[2:0]$ priamo kódujú **prvý registrový argument** (Rd alebo Rs1), čo umožňuje adresovať plnú sadu 8 registrov R0–R7 bez dodatočnej réžie dekódovania.
- V inštrukciách vyžadujúcich voľbu hardvérového režimu alebo selektora pamäte/špeciálnych registrov (LMR, LRM, MSP, JMP, JCR, JNZ, JZ), bity $[2:0]$ slúžia ako konfigurátor režimu (CFG / MOD / SRC / DST).
- V bezargumentových riadiacich inštrukciách (NOP, HALT, CLI, STI, RET) tieto bity obsahujú `000`.

4.2 Trieda I: Jednobajtové inštrukcie (1 bajt, 0 doplňujúcich oktetov)

Riadiace inštrukcie (NOP, HALT, CLI, STI, RET) a unárne registrové operácie (NOT, INC, DEC) sa načítavajú presne za jeden takt výberu. Druhý bajt zo zbernice pamäte sa vôbec nevyžaduje:

- Pri bezargumentových inštrukciách spodné 3 bity obsahujú `000`.
- Pri unárnych inštrukciách (`INC Rd`, `DEC Rd`, `NOT Rd`) spodné 3 bity (`[2:0]`) priamo obsahujú index cieľového registra Rd (R0–R7, jediný argument).



Oktet 0: Fixná dĺžka 1 bajt

4.3 Trieda II: Trojoperandové aritmetické inštrukcie (2 bajty)

Výpočtové inštrukcie (ADD, SUB, MUL, DIV, MOD) striktne realizujú architektonickú paradigmu **„najprv cieľ, potom operandy“**:

$$\texttt{ADD Rd, Rs1, Rs2} \quad \Longrightarrow \quad \texttt{Rd} \leftarrow \texttt{Rs1} + \texttt{Rs2} + \texttt{ALU_RESIDUAL}$$

$$\texttt{MOD Rd, Rs1, Rs2} \quad \Longrightarrow \quad \texttt{Rd} \leftarrow \texttt{Rs1} \pmod{\texttt{Rs2}}$$

Prvý argument inštrukcie (cieľový register určenia Rd) sa umiestňuje priamo do spodných 3 bitov prvého oktetu, čo umožňuje plné 3-bitové adresovanie (R0–R7) pre všetky tri operandy:

- **Oktet 0:** bity $[7:3]$ — kód operácie (5 bitov); bity $[2:0]$ — cieľový register určenia Rd (3 bity, R0–R7, 1. argument).
- **Oktet 1:** bity $[7:5]$ — prvý zdrojový operand Rs1 (3 bity, R0–R7); bity $[4:2]$ — druhý zdrojový operand Rs2 (3 bity, R0–R7); bity $[1:0]$ — rezerva (`00`).



4.4 Trieda III: Dvojoperandové logické inštrukcie (2 bajty)

Logické operácie (AND, OR, NAND, NOR, XOR, XNOR) majú presne dva operandy:

$$\texttt{AND Rd, Rs} \quad \Longrightarrow \quad \text{Rd} \leftarrow \text{Rd AND Rs}$$

- **Oktet 0:** bity $[7:3]$ — kód operácie; bity $[2:0]$ — cieľový register-príjemca Rd (R0–R7, 1. argument).
- **Oktet 1:** bity $[7:5]$ — druhý register-operand Rs (R0–R7); bity $[4:0]$ — vyhradené (00000_2).



4.5 Trieda IV: Medziregistrový presun LRR (2 bajty)

Inštrukcia LRR Rd, Rs kopíruje dáta zo zdroja (bod B, Rs) do príjemcu (bod A, Rd):

$$\texttt{LRR Rd, Rs} \quad \Longrightarrow \quad \text{Rd} \leftarrow \text{Rs}$$

Presun sa vykonáva cez priamu obchádzkovú vetvu **MUX 1** $\rightarrow$ **MUX 2** bez zaťaženia ALU počas jedného mikrotaktu, bez ovplyvnenia registra príznakov FREG. Formát oktetov: Oktet 0 obsahuje opkód `00111` a cieľový register Rd (`[2:0]`), Oktet 1 obsahuje zdrojový register Rs (`[7:5]`) a rezervu (`[4:0]` = `00000`).

4.6 Trieda V: Hardvérový komparátor CMP (2 bajty)

Inštrukcia CMP Rs1, Rs2 porovnáva hodnoty registrov bez narušenia ich obsahu:

- **Oktet 0:** kód operácie `01000`, bity $[2:0]$ určujú prvý zdrojový operand Rs1 (1. argument).
- **Oktet 1:** bity $[7:5]$ určujú druhý zdrojový operand Rs2; bity $[4:0]$ obsahujú `00000`.

Výsledok porovnania sa ukladá do špecializovaného 2-bitového poľa registra stavu FREG.CMP (bity $[5:4]$):

Hodnota FREG.CMP	Podmienka	Architektonický význam
`01` (01_2)	Rs1 $<$ Rs2	Prvý operand je ostro menší ako druhý
`10` (10_2)	Rs1 $=$ Rs2	Hodnoty operandov sú zhodné
`11` (11_2)	Rs1 $>$ Rs2	Prvý operand je ostro väčší ako druhý
`00` (00_2)	Predvolené / Reset	Nič sa neporovnávalo / chyba komparátora

4.7 Trieda VI: Hardvérový cyklický posun na D-klopných obvodoch (BSL, BSR)

Inštrukcie posunu BSL Rd, shift a BSR Rd, shift sú v kremíku realizované ako **iteratívny 16-bitový posuvný register na D-klopných obvodoch** (≈ 300 tranzistorov namiesto $\approx 30\,000$ pri maticovom barelovom posúvači):

- Posun prebieha hardvérovo cyklom takt-za-taktom (`for` v hardvéri): posun o 16 bitov vyžaduje presne o 15 taktov viac než posun o 1 bit.

- Oktet 0: horných 5 bitov tvorí opkód (BSL: 11000, BSR: 11001), a spodné 3 bity ([2:0]) priamo určujú posúvaný register Rd.
- Oktet 1: celý bajt ([7:0]) udáva počet taktov posunu shift (0–16).
- Správanie bitov je riadené registrom FREG: pri SMOD $= 1$ je aktívna hardvérová rotácia cez vytlačený bit; pri SMOD $= 0$ sa prázdne bity plnia hodnotou z FREG.DBIT.



4.8 Trieda VII: Operácie so zásobníkom (PUSH, POP)

Inštrukcie zásobníka ukladajú register do pamäte na základe 32-bitového ukazovateľa SREG:

- **Oktet 0:** kód operácie (PUSH: 10110, POP: 10111), bity $[2:0]$ priamo určujú register operandu (Rs pri uložení, Rd pri výbere, 1. argument).
- **Oktet 1:** bity $[7:0]$ obsahujú 00000000_2 (rezerva).

4.9 Trieda VIII: Manipulácia so špeciálnymi registrami (MSP, Opkód 26)

Inštrukcia MSP (Opkód 11010, 26_{10}) poskytuje modulárne rozhranie na konfiguráciu všetkých systémových, adresných a ochranných prvkov jadra: FREG, PTREG, SREG, ako aj 32 systémových vektorových registrov pre prerušenia a Zóny Privilégií ($ir0..$ir15, $pv0..$pv15). Dĺžka inštrukcie je v závislosti od režimu od 2 do 6 bajtov.

4.9.1 Formát 8.1: Načítanie do príznakového registra FREG (MOD = 0, REG = 00)

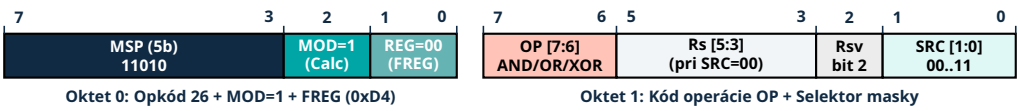
Príkaz MSP.ld $freg, <src> vykonáva priamu inicializáciu príznakov.



- SRC $= 00_2$ (2 bajty): načítanie z registra GPR Rs (MSP.ld $freg, $rs).
- SRC $= 01_2$ (4 bajty): priama konštanta imm16 v oktetoch 2–3 (MSP.ld $freg, 0x0000).
- SRC $= 10_2$ (2 bajty): čítanie z pamäte cez ukazovateľ [$ptreg] (MSP.ld $freg, [$ptreg]).
- SRC $= 11_2$ (4 bajty): čítanie z pamäte na priamej 16-bitovej adrese [addr16] (MSP.ld $freg, [0x1000]).

4.9.2 Formát 8.2: Bitová logika FREG (MOD = 1, REG = 00)

Inštrukcie MSP.and, MSP.or, MSP.xor, MSP.xnor aplikujú bitovú masku na príznaky.



- **Kód operácie OP**$[7:6]$**:** 00_2 = AND, 01_2 = OR, 10_2 = XOR, 11_2 = XNOR.
- **Hardvérová ochrana režimu supervízora (PRV):** Procesor hardvérovo štartuje pri zapnutí napájania a resete v režime správcu (PRV=1). Úvodný kód (zavádzač/jadro OS) nakonfiguruje vektory ($ir0..$ir15) a hranice Zón Privilégií ($pv0..$pv15), načo zníži oprávnenia na PRV=0 a

odovzdá riadenie používateľskému kódu. V používateľskom režime (PRV=0) je aktivácia PRV=1 a úprava registrov zón hardvérovo povolená výhradne vtedy, ak sa aktuálne PC nachádza striktne vo vnútri jednej z 8 Zón Privilégií ($\exists k \in [0..7] : \$pv_{2k} \leq \text{PC} \leq \$pv_{2k+1}$). **Dôležité architektonické pravidlo:** pri opustení Zóny Privilégií sa režim supervízora PRV=1 automaticky **neresetuje** — operačný systém/program je sám zodpovedný za jeho explicitné vynulovanie pred odovzdaním riadenia používateľovi.

4.9.3 Formát 8.3: Načítanie ukazovateľov PTREG a SREG (MOD = 0, REG = 01 / 10)

Príkazy `MSP.ld $ptreg` (REG $= 01_2$, opkód `0xD1`) a `MSP.ld $sreg` (REG $= 10_2$, opkód `0xD2`) vykonávajú priamu inicializáciu 32-bitových lineárnych ukazovateľov.

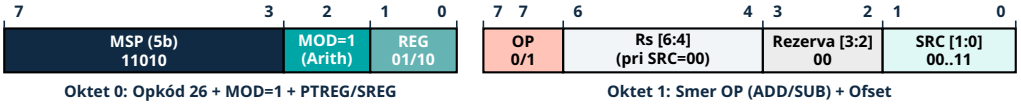


Oktet 0: Opkód 26 + MOD=0 + PTREG/SREG — Oktet 1: Dvojica GPR alebo selektor zdroja (2–6 bajtov)

- SRC $= 00_2$ (2 bajty): zloženie adresy z dvojice GPR Rd_hi:Rs_lo (`MSP.ld $ptreg, [$0:$1]`).
- SRC $= 01_2$ (4–6 bajtov): priama adresa `imm32` (6 bajtov) alebo `imm16` (4 bajty).
- SRC $= 10_2$ (**2 bajty**): **natívne 1-taktové dereferencovanie ukazovateľa** `[$ptreg]` (`PTREG <- MEM[PTREG]`).
- SRC $= 11_2$ (4–6 bajtov): načítanie ukazovateľa z pamäte z priamej adresy `[addr]` (`MSP.ld $ptreg, [0xA000]`).

4.9.4 Formát 8.4: Aritmetika posunu PTREG a SREG (MOD = 1, REG = 01 / 10)

Príkazy `MSP.add` (OP $= 0$) a `MSP.sub` (OP $= 1$) vykonávajú rýchle hardvérové posúvanie ukazovateľov zásobníka a dátových vyrovnávacích pamätí.

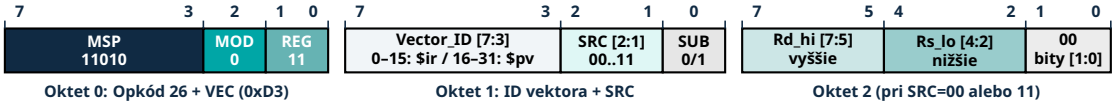


Oktet 0: Opkód 26 + MOD=1 + PTREG/SREG — Oktet 1: Smer OP (ADD/SUB) + Ofset

- OP $= 0$: ADD (+posun k ukazovateľu), OP $= 1$: SUB (-posun od ukazovateľa).
- Zdroj posunu: register GPR SRC $= 00_2$ (2 bajty), priama konštanta SRC $= 01_2$ (4 bajty), čítanie posunu cez SRC $= 10_2$ (`[$ptreg]`, 2 bajty) alebo SRC $= 11_2$ (`[addr16]`, 4 bajty).

4.9.5 Formát 8.5: Načítanie 32-bitových vektorových registrov (MOD = 0, REG = 11)

Príkaz `MSP.ld` načítava 32-bitové adresy do jedného z 32 vektorových registrov: `$ir0..$ir15` (vektory prerušení, ID 0–15) a `$pv0..$pv15` (hranice Zón Privilégií, ID 16–31).



Oktet 0: Opkód 26 + VEC (0xD3) — Oktet 1: ID vektora + SRC — Oktet 2 (pri SRC=00 alebo 11)

- **Variant 5.1 (SRC $= 00_2$, SUB $= 0$, 3 bajty):** načítanie vektora z dvojice GPR Rd_hi:Rs_lo (`MSP.ld $ir0, [$0:$1]`).
- **Variant 5.2 (SRC $= 01_2$, SUB $= 0$, 6 bajtov):** priamy 32-bitový vektor adresy `imm32` v oktetoch 2–5 (`MSP.ld $pv0, 0x00008000`).
- **Variant 5.3 (SRC $= 10_2$, SUB $= 0$, 2 bajty):** čítanie vektora z pamäte cez ukazovateľ `[$ptreg]` (`MSP.ld $ir1, [$ptreg]`).

- **Variant 5.4 (SRC $= 11_2$, SUB $= 0$, 3 bajty):** čítanie vektora z pamäte cez 32-bitový ukazovateľ v dvojici GPR `[$rd:$rs]` (`MSP.ld $ir2, [$0:$1]`).
- **Variant 5.5 (SRC $= 11_2$, SUB $= 1$, 6 bajtov):** čítanie vektora z pamäte z priamej 32-bitovej adresy `[addr32]` (`MSP.ld $ir0, [0x00005000]`).

4.10 Trieda IX: Inštrukcie vetvenia a skokov (2–3 bajty)

Skokové inštrukcie (JMP, JCR, JNZ, JZ) podporujú tri navzájom sa vylučujúce adresné režimy určené poľom CFG (bity $[2:0]$ oktetu 0):

- **MOD = 0 (000): Priama 16-bitová adresa (dĺžka 3 bajty).** Za oktetom 0 nasledujú oktety 1 a 2 tvoriace absolútnu 16-bitovú adresu skoku v aktuálnom 64-KB segmente.
- **MOD = 1 (001): Nepriamy registrový skok (dĺžka 2 bajty).** Oktet 1 určuje register Rs (bity $[7:5]$), ktorého hodnota sa načíta do PC.
- **MOD = 2 (010): Rozšírený 24-bitový skok cez PTREG.** Skok sa vykoná na fyzickú 24-bitovú adresu zo systémového registra PTREG.

4.11 Trieda X: Inštrukcie výmeny s pamäťou LMR a LRM (2–4 bajty)

Inštrukcie načítania (LMR) a uloženia (LRM) sa vyznačujú najvyššou adaptabilitou. Konfiguračné pole oktetu 0 (`[2:0]`) kóduje:

- Bit $[2]$: veľkosť operandu SZ ($0 = 8$ bitov / bajt, $1 = 16$ bitov / slovo).
- Bity $[1:0]$: zdroj/cieľ SRC / DST:
  - **0: Priame dáta (Inline Data):** 2 bajty pri SZ $= 0$ (bajt za inštrukciou) alebo 3 bajty pri SZ $= 1$ (16-bitové slovo za inštrukciou).
  - **1: Priamy ukazovateľ (Imm Pointer):** 4 bajty. Oktet 1 určuje register, a oktety 2–3 uchovávajú 16-bitovú adresu dát.
  - **2: Systémový 24-bitový ukazovateľ PTREG:** 2 bajty. Oktet 1 určuje cieľový register. Pri aktívnom príznaku FREG.AIP sa ukazovateľ PTREG hardvérovo inkrementuje za 0 dodatočných taktov.
  - **3: Zložená 24-bitová adresa z dvojice registrov:** 2 bajty. Oktet 1 obsahuje bity $[7:5]$ (Arg1 / vyššie bity adresy) a bity $[4:2]$ (Arg2 / nižších 16 bitov adresy). V inštrukcii LMR sa register Arg1 po načítaní prepíše získanými dátami.

5 Prehľadná tabuľka inštrukcií jadra LIM M2

Č.	Kód	Mnemonic	Trieda	Funkčný účel
0	00000	NOP	Riadenie	Prázdna operácia a vyprázdnenie zvyškového registra ALU
1	00001	ADD	Aritmetika	Súčet s automatickým reťazením prenosu cez zvyšok ALU
2	00010	SUB	Aritmetika	Odčítanie s automatickým reťazením výpožičky cez zvyšok ALU
3	00011	MUL	Aritmetika	Násobenie so zachytením vyšších bitov vo zvyšku ALU
4	00100	DIV	Aritmetika	Delenie s uložením zvyšku do vnútorného registra ALU
5	00101	LMR	Pamäť	Načítanie z pamäte/vstupu do registra (konfig zdroja 0–3, veľkosť 8/16 bit)
6	00110	LRM	Pamäť	Zápis dát z registra do pamäte/výstupu (konfig cieľa 0–3, veľkosť 8/16 bit)
7	00111	LRR	Presun	Priame kopírovanie medzi registrami cez obchádzkový MUX

Č.	Kód	Mnemonic	Trieda	Funkčný účel
8	01000	CMP	Porovnanie	Hardvérové porovnanie operandov bez zmeny dát
9	01001	JMP	Skok	Nepodmienený skok (priamy, nepriamy, 24-bitový)
10	01010	JCR	Skok	Podmienený skok pri nastavenom príznaku prenosu CF
11	01011	JNZ	Skok	Podmienený skok pri nenulovom výsledku (ZF = 0)
12	01100	JZ	Skok	Podmienený skok pri nulovom výsledku (ZF = 1)
13	01101	AND	Logika	Bitový logický súčin (AND)
14	01110	OR	Logika	Bitový logický súčet (OR)
15	01111	NAND	Logika	Bitová negácia logického súčinu (NAND)
16	10000	NOR	Logika	Bitová negácia logického súčtu (NOR)
17	10001	NOT	Logika	Bitová inverzia operandu (jednotkový doplnok)
18	10010	XOR	Logika	Bitová neverzia (XOR)
19	10011	XNOR	Logika	Bitová ekvivalencia
20	10100	INC	Aritmetika	Hardvérová inkrementácia operandu o 1
21	10101	DEC	Aritmetika	Hardvérová dekrementácia operandu o 1
22	10110	PUSH	Zásobník	Uloženie slova na vrchol zásobníka
23	10111	POP	Zásobník	Výber slova z vrcholu zásobníka do registra
24	11000	BSL	Posuny	Barelový posun doľava s konfigurovateľným vstupom
25	11001	BSR	Posuny	Barelový posun doprava s ukladaním do DBIT
26	11010	MSP	Systém	Manipulácia so špeciálnymi registrami (FREG, PTREG, SREG)
27	11011	MOD	Aritmetika	Výpočet zvyšku po celočíselnom delení (modulo)
28	11100	INTR / INT	Systém	Softvérové prerušenie (2 bajty, bity [3:0] 2. bajtu — vektor 0..15)
29	11101	CALL	Skoky	Volanie podprogramu s uložením 32-bitovej návratovej adresy na zásobník
30	11110	RET	Skoky	Návrat z podprogramu s obnovením 32-bitového PC zo zásobníka SREG
31	11111	HALT	Riadenie	Hardvérové zastavenie procesora (HLT)

6 Detailná špecifikácia inštrukčného súboru

Nasledujúce strany obsahujú úplné architektonické profily všetkých 32 operačných kódov procesora LIM M2.

NOP

Hardvérová prázdna operácia a vyprázdnenie aritmetickej reťaze

00000
Riadenie

Mnemonic	NOP
Opcode	`00000` (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Riadenie
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Hardvérová prázdna operácia a vyprázdnenie aritmetickej reťaze.

Architektonická sémantika

```
Signatúra:  NOP
Operácia:  PC <- PC + 1, ALU_RESIDUAL <- 0, FREG.CF <- 0, FREG.OF <- 0
Príklad:  NOP ; Reset reťazca prenosu ALU
```

Konfigurácia a operandy



Oktet 0 (1 bajt, 0 doplň. bajtov)

Inštrukcia pevnej dĺžky (1 bajt). Bez operandov; pole cfg obsahuje `000` (rezervované na kalibráciu časovania). Druhý bajt zo zbernice sa nenačítava.

Vplyv na príznakový register (FREG)

Vynuluje príznaky prenosu (CF), pretečenia (OF) a výpožičky; ostatné príznaky zachováva.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Slúži na synchronizáciu taktov a hardvérové vyprázdnenie (Flush) vnútorného zvyškového registra ALU, čím prerušuje reťazenie pred novými nezávislými výpočtami.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

ADD

Sčítanie s automatickým reťazením prenosu

00001
Aritmetika

Mnemonic	ADD
Opcode	00001 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Aritmetika
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

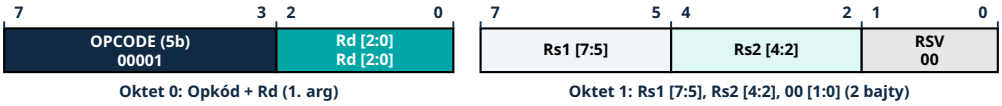
Účel inštrukcie

Sčítanie s automatickým reťazením prenosu.

Architektonická sémantika

```
Signatúra:  ADD arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 + arg2
Operácia:  sum <- arg1 + arg2 + ALU_RESIDUAL
arg0 <- sum[15:0], ALU_RESIDUAL <- (sum > 0xFFFF ? 1 :  0)
Príklad:  ADD R0, R1, R2 ; R0 = R1 + R2 (registre R0–R7)
```

Konfigurácia a operandy



Oktet 0: Opkód + Rd (1. arg)

Oktet 1: Rs1 [7:5], Rs2 [4:2], 00 [1:0] (2 bajty)

Oktet 0: kód operácie 00001, bity $[2:0]$ určujú cieľový register Rd (R0–R7, 1. argument).
Oktet 1: bity $[7:5]$ určujú prvý sčítanec Rs1 (R0–R7); bity $[4:2]$ určujú druhý sčítanec Rs2 (R0–R7); bity $[1:0]$ sú rezerva (00). Dĺžka inštrukcie: presne 2 bajty.

Vplyv na príznakový register (FREG)

Nastavuje príznak prenosu CF a pretečenia OF pri prekročení $2^{16} - 1$; aktualizuje ZF podľa dst.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.

4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Podporuje priebežnú viacbajtovú aritmetiku (32, 48, 64+ bitov). Ak súčet prekročí 16 bitov ($> 2^{16} - 1$), prenos (+1) sa zachová vo vnútornom registri ALU a automaticky sa pripočíta pri nasledujúcej operácii ADD bez nutnosti inštrukcie ADC. Spodných 16 bitov sa deterministicky zapíše do dst. Reťazenie sa preruší inštrukciou NOP alebo novou matematickou operáciou.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

SUB

Odčítanie s automatickým reťazením výpožičky

00010
Aritmetika

Mnemonic	SUB
Opcode	`00010` (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Aritmetika
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

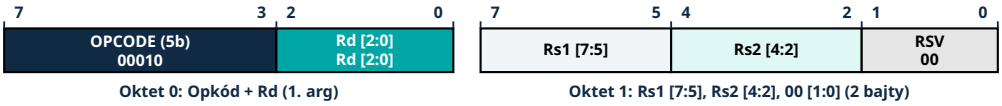
Účel inštrukcie

Odčítanie s automatickým reťazením výpožičky.

Architektonická sémantika

```
Signatúra:  SUB arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 - arg2
Operácia:  diff <- arg1 - arg2 - ALU_RESIDUAL
arg0 <- diff[15:0], ALU_RESIDUAL <- (diff < 0 ?  1 :  0)
Príklad:  SUB R0, R1, R2 ; R0 = R1 - R2 (registre R0–R7)
```

Konfigurácia a operandy



Oktet 0: Opkód + Rd (1. arg)

Oktet 1: Rs1 [7:5], Rs2 [4:2], 00 [1:0] (2 bajty)

Oktet 0: kód operácie `00010`, bity $[2:0]$ určujú cieľový register Rd (R0–R7, 1. argument).
Oktet 1: bity $[7:5]$ určujú menšenec `Rs1` (R0–R7); bity $[4:2]$ určujú menšiteľ `Rs2` (R0–R7); bity $[1:0]$ sú rezerva (`00`). Dĺžka inštrukcie: presne 2 bajty.

Vplyv na príznakový register (FREG)

Pri zápornom rozdiele (< 0) nastavuje príznak výpožičky (Borrow / CF) a OF; aktualizuje ZF.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.

4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Umožňuje kontinuálne viacbajtové odčítanie. Pri zápornom rozdiele (< 0) sa výpožička (-1) uloží do vnútorného registra ALU a pri ďalšej inštrukcii SUB sa automaticky odpočíta 1 od vyššieho slova. Výsledok spodného slova sa deterministicky uloží do dst. Na začatie nového nezávislého odčítania sa reťazenie preruší pomocou NOP.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

MUL

Celočíselné násobenie so zachovaním hornej časti v registri ALU

00011
Aritmetika

Mnemonic	MUL
Opcode	00011 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Aritmetika
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Celočíselné násobenie so zachovaním hornej časti v registri ALU.

Architektonická sémantika

```
Signatúra:  MUL arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 * arg2
Operácia:  prod <- arg1 * arg2
arg0 <- prod[15:0], ALU_RESIDUAL <- prod[31:16]
Príklad:  MUL R0, R1, R2 ; R0 = R1 * R2 (horná časť v ALU_RESIDUAL)
```

Konfigurácia a operandy



Oktet 0: Opkód + Rd (1. arg)

Oktet 1: Rs1 [7:5], Rs2 [4:2], 00 [1:0] (2 bajty)

Oktet 0: kód operácie 00011, bity $[2:0]$ určujú cieľový register Rd (R0–R7, 1. argument).
Oktet 1: bity $[7:5]$ určujú prvý činiteľ Rs1 (R0–R7); bity $[4:2]$ určujú druhý činiteľ Rs2 (R0–R7); bity $[1:0]$ sú rezerva (00). Horných 16 bitov súčinu sa automaticky zachytáva v registri ALU_RESIDUAL. Dĺžka inštrukcie: 2 bajty.

Vplyv na príznakový register (FREG)

Aktualizuje ZF; nastavuje OF/CF, ak je horných 16 bitov súčinu (prod[31:16]) nenulových.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.

2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Pri 32-bitovom súčine smeruje spodných 16 bitov do cieľového registra dst, kým horných 16 bitov sa uloží do vnútorného zvyškového registra ALU (ALU Residual). Pri nasledujúcom sčítaní sa tento zvyšok automaticky pripočíta (podpora MAC a viacbajtových čísiel). Hornú polovicu možno získať aj sčítaním registra so sebou samým po vynulovaní. Inštrukcia NOP register vyčistí.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

DIV

Celočíselné delenie so zachovaním zvyšku vo vnútornom registri ALU

00100
Aritmetika

Mnemonic	DIV
Opcode	00100 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Aritmetika
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

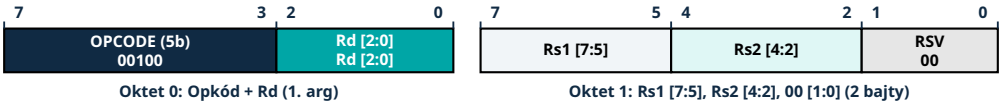
Účel inštrukcie

Celočíselné delenie so zachovaním zvyšku vo vnútornom registri ALU.

Architektonická sémantika

```
Signatúra:  DIV arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 / arg2
Operácia:  arg0 <- arg1 / arg2
ALU_RESIDUAL <- arg1 % arg2
Príklad:  DIV R0, R1, R2 ; R0 = R1 / R2 (zvyšok modulo v ALU_RESIDUAL)
```

Konfigurácia a operandy



Oktet 0: kód operácie 00100, bity $[2:0]$ určujú cieľový register Rd (R0–R7, 1. argument).
Oktet 1: bity $[7:5]$ určujú delenec Rs1 (R0–R7); bity $[4:2]$ určujú deliteľ Rs2 (R0–R7); bity $[1:0]$ sú rezerva (00). Zvyšok po delení (modulo) sa automaticky ukladá do registra ALU_RESIDUAL. Dĺžka inštrukcie: 2 bajty.

Vplyv na príznakový register (FREG)

Pri delení nulou sa nastavuje príznak chyby EM (Error Math) bez zablokovania procesora; aktualizuje ZF.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Podiel sa zapíše do cieľového registra dst, pričom zvyšok po delení (modulo) sa zachová vo vnútornom registri ALU pre ďalšie operácie alebo vymazanie inštrukciou NOP.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

LMR

Načítanie dát z pamäte a externých zariadení do registrov

00101
Pamäť

Mnemonic	LMR
Opcode	00101 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Pamäť
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Načítanie dát z pamäte a externých zariadení do registrov.

Architektonická sémantika

```
Signatúra:  LMR arg0(REG), <SRC_MODE> :  arg0 = MEM[...]
Režimy:  SRC=0:  LMR R0, [PC+] ; Imm Data (1–2 bajty)
SRC=1:  LMR R0, [[PC+]] ; Imm Ukazovateľ
SRC=2:  LMR R0, [PTREG] ; Systémový ukazovateľ (AIP)
SRC=3:  LMR R0(R_hi), R1(R_lo) ; 24-bitová adresa z páru
Operácia:  arg0 <- MEM[EA] (horné bity adresy sa prepíšu dátami)
Príklad:  LMR R0, R1 ; EA=(R0«16)|R1, načítanie MEM[EA] do R0
```

Konfigurácia a operandy



Oktet 0: bit $[8]$ (SZ) určuje šírku dát ($0 = 1$ bajt, $1 = 2$ bajty / slovo); bity $[10:9]$ (SRC) volia zdroj dát (0 — priame dáta za kódom inštrukcie, 1 — priamy ukazovateľ za inštrukciou, 2 — nepriamo cez 24-bitový ukazovateľ PTREG, 3 — nepriamo cez dvojicu používateľských registrov).
Oktet 1: bity $[7:5]$ definujú register Arg1 (R_hi / cieľ), bity $[4:2]$ definujú register Arg2 (R_lo). V režime SRC $= 3$ používateľ zvolí dva 16-bitové registre tvoriace 32-bitové číslo, z ktorého sa na adresovanie použije 24 bitov: Arg1 nesie horné bity adresy a Arg2 dolných 16 bitov. Register Arg1 je zároveň cieľovým registrom: po výbere z pamäte sa horné bity adresy v Arg1 prepíšu načítanými dátami.

Vplyv na príznakový register (FREG)

Všetky príznaky v stavovom registri FREG zostávajú zachované bez zmeny.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Inštrukcia LMR predstavuje kľúčovú vstupnú bránu pre prenos externých dát a pamäte do registrového poľa procesora LIM M2 ($R0$–$R7$). Architektúra integruje štyri ortogonálne režimy výberu s natívnym 24-bitovým adresovaním:
1. V režime SRC $= 0$ procesor číta literál priamo z toku kódu (1 alebo 2 bajty) s automatickým posunom čítača inštrukcií PC.
2. V režime SRC $= 1$ sa za inštrukciou načíta absolútny ukazovateľ, podľa ktorého sa vyberú dáta z pamäte.
3. V režime SRC $= 2$ prebieha prístup na 24-bitovú fyzickú adresu z registra PTREG. Pri aktívnom príznaku autoincrementu FREG.AIP (bit 12) sa register PTREG automaticky zvýši o veľkosť výberu ($+1$ alebo $+2$), čo umožňuje prúdové načítanie polí za jediný takt.
4. V režime SRC $= 3$ používateľ zvolí dva 16-bitové registre (Arg1 a Arg2), ktoré spolu vytvárajú 32-bitové číslo, z ktorého sa v praxi adresovania využíva dolných 24 bitov. Používateľ určí, že horné bity adresy sú v Arg1 a dolných 16 bitov v Arg2. V tomto režime sa register Arg1 s hornými bitmi adresy stáva zároveň cieľovým registrom, do ktorého sa zapíšu načítané dáta (teda horné bity adresy v Arg1 sa prepíšu dátami, kým Arg2 uchováva dolných 16 bitov adresy).

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

LRM

Zápis dát z registrov do pamäte a externých zariadení

00110
Pamäť

Mnemonic	LRM
Opcode	00110 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Pamäť
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Zápis dát z registrov do pamäte a externých zariadení.

Architektonická sémantika

```
Signatúra:  LRM arg0(REG), <DST_MODE> :  MEM[...]  = arg0
Režimy:  DST=0:  LRM R0, [PC+] ; Vložená pamäť za kódom
DST=1:  LRM R0, [[PC+]] ; Imm Ukazovateľ
DST=2:  LRM R0, [PTREG] ; Systémový ukazovateľ (AIP)
DST=3:  LRM R0(R_hi), R1(R_lo) ; 24-bitová adresa z páru
Operácia:  MEM[EA] <- arg0[15:0] (alebo dolný bajt pri SZ=0)
Príklad:  LRM R0, R1 ; EA=(R0«16)|R1, zápis R0 do MEM[EA]
```

Konfigurácia a operandy



Oktet 0: bit $[8]$ (SZ) určuje šírku dát ($0 = 1$ bajt, $1 = 2$ bajty / slovo); bity $[10:9]$ (DST) volia cieľové umiestnenie dát (0 — zápis priamo za kód inštrukcie, 1 — zápis podľa ukazovateľa za inštrukciou, 2 — nepriamo cez 24-bitový ukazovateľ PTREG, 3 — nepriamo cez dvojicu registrov používateľa).
Oktet 1: bity $[7:5]$ definujú zdrojový register Arg1 (Rs / R_hi), bity $[4:2]$ definujú register Arg2 (R_lo). V režime DST $= 3$ dva registre tvoria 32-bitové číslo, z ktorého sa použije 24 bitov adresy (Arg1 — horné bity adresy a zdroj dát, Arg2 — dolných 16 bitov). Prepísanie existujúcich dát na cieľovej adrese je plnou zodpovednosťou programátora.

Vplyv na príznakový register (FREG)

Všetky príznaky v stavovom registri FREG zostávajú zachované bez zmeny.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Inštrukcia LRM predstavuje symetrický mechanizmus prenosu registrových dát procesora LIM M2 ($R0$–$R7$) do pamäte a periférnych obvodov. Architektúra integruje štyri ortogonálne režimy adresovania:
1. V režime DST $= 0$ procesor zapisuje 1 alebo 2 bajty priamo do pamäte za inštrukciou (PC) s posunom čítača inštrukcií. Režim slúži pre dynamické buffery a samomodifikujúci sa kód; prepísanie dát je na zodpovednosti programátora.
2. V režime DST $= 1$ sa za inštrukciou načíta 16-bitový absolútny ukazovateľ, na ktorý sa dáta uložia.
3. V režime DST $= 2$ zápis smeruje na 24-bitovú systémovú adresu registra PTREG. Pri aktívnom príznaku FREG.AIP (bit 12) sa register PTREG automaticky zvýši o $+1$ alebo $+2$, čo umožňuje prúdový DMA výstup do pamäte za jediný takt.
4. V režime DST $= 3$ sa využíva dvojica 16-bitových registrov: Arg1 a Arg2 tvoria 32-bitové číslo, ktorého dolných 24 bitov určuje fyzickú adresu (Addr$[23:0]$). Dáta z Arg1 sa zapíšu na vypočítanú adresu, čo sprístupňuje celých 16 MB priestoru.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

LRR

Medziregistrový presun (kopírovanie z bodu B do bodu A)

00111
Presun

Mnemonic	LRR
Opcode	00111 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Presun
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Medziregistrový presun (kopírovanie z bodu B do bodu A).

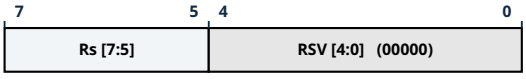
Architektonická sémantika

```
Signatúra:  LRR arg0(REG), arg1(REG) : arg0 = arg1
Operácia:  arg0 <- arg1 (priame kopírovanie z bodu B do bodu A)
Príklad:  LRR R0, R1 ; R0 = R1 (hodnota R1 sa skopíruje do R0)
```

Konfigurácia a operandy



Oktet 0: Opkód + Rd



Oktet 1: Rs [7:5] (Dĺžka: 2 bajty)

Oktet 0: kód operácie 00111_2 (0x07), bity $[2:0]$ určujú cieľový register-príjemcu Rd (R0–R7, bod A, 1. argument).
Oktet 1: bity $[7:5]$ určujú zdrojový register Rs (R0–R7, bod B); bity $[4:0]$ obsahujú 00000_2 (rezerva).
Pri vykonaní sa hodnota Rs prenesie do Rd za 1 mikrocyklus. Dĺžka inštrukcie: 2 bajty.

Vplyv na príznakový register (FREG)

Všetky príznaky zostávajú zachované.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).

3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Inštrukcia LRR predstavuje základnú operáciu presunu dát medzi registrami všeobecného určenia procesora ($R0$–$R7$). Vykonáva prenos zo zdroja (bod B) do cieľa (bod A). Presun sa uskutočňuje cez priamy hardvérový premosťovací kanál **MUX 1** $\rightarrow$ **MUX 2** bez zaťažovania ALU a bez vplyvu na stav príznakov registra FREG.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

CMP

Hardvérové porovnanie operandov

01000
Porovnanie

Mnemonic	CMP
Opcode	01000 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Porovnanie
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

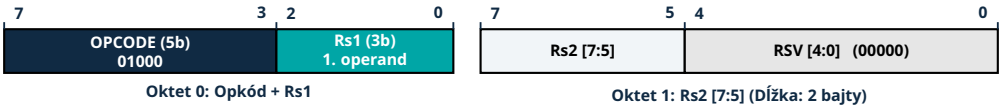
Účel inštrukcie

Hardvérové porovnanie operandov.

Architektonická sémantika

```
Signatúra:  CMP arg0(REG), arg1(REG) : compare(arg0, arg1)
Operácia:  if arg0 < arg1 then FREG.CMP <- 01b (menšie)
if arg0 == arg1 then FREG.CMP <- 10b (rovné)
if arg0 > arg1 then FREG.CMP <- 11b (väčšie)
(stav 00b – počiatočný reset / neporovnané / chyba)
Príklad:  CMP R0, R1 ; 01:  R0<R1, 10:  R0==R1, 11:  R0>R1 (00:  reset/chyba)
```

Konfigurácia a operandy



Oktet 0: kód operácie 01000, bity $[2:0]$ určujú prvý operand Rs1 (1. argument).
Oktet 1: bity $[7:5]$ určujú druhý operand Rs2; bity $[4:0]$ obsahujú 00000 (rezerva).
Výsledok hardvérového porovnania sa ukladá do poľa FREG.CMP (bity $[5:4]$): 01 (Rs1 $<$ Rs2), 10 (Rs1 $=$ Rs2), 11 (Rs1 $>$ Rs2), 00 (reset / neporovnávané / chyba). Hodnoty registrov sa nemenia.
Dĺžka inštrukcie: 2 bajty.

Vplyv na príznakový register (FREG)

Aktualizuje 2-bitové pole FREG.CMP (bity [5:4]) a príznaky ZF, CF, OF bez zmeny registrov.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Umožňuje rýchle podmienené vetvenie.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

JMP

Nepodmienený skok

01001
Skok

Mnemonic	JMP
Opcode	01001 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Skok
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Nepodmienený skok.

Architektonická sémantika

```
Signatúra:  JMP target(IMM|REG) : PC = target
Operácia:  PC <- target
Príklad:  JMP R0 ; Bezpodmienečný skok na adresu v R0
```

Konfigurácia a operandy



Oktet 0: kód operácie 01001, bity $[2:0]$ (MOD): 000 — 16-bitová priama adresa (dĺžka 3 bajty, oktety 1–2 obsahujú adresu); 001 — nepriamy registrový skok (dĺžka 2 bajty, oktet 1 bity $[7:5]$ určujú register Rs); 010 — rozšírený 24-bitový skok cez PTREG.

Vplyv na príznakový register (FREG)

Príznaky zostávajú nezmenené.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Atomické načítanie Program Counter v minimálnom počte cyklov.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

JCR

Podmienený skok pri prenose

01010
Skok

Mnemonic	JCR
Opcode	01010 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Skok
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Podmienený skok pri prenose.

Architektonická sémantika

```
Signatúra:  JCR target(IMM|REG) : if FREG.CF == 1 then PC = target
Operácia:  if FREG.CF == 1 then PC <- target
Príklad:  JCR R2 ; Skok na adresu R2, ak je nastavený Carry
```

Konfigurácia a operandy



Oktet 0: kód operácie 01010, bity $[2:0]$ (MOD): 000 (16-bitová adresa Imm, 3 bajty), 001 (nepriamo cez Rs, 2 bajty), 010 (24-bitový cez PTREG). Skok sa vykoná iba pri nastavenom príznaku prenosu FREG.CF = 1.

Vplyv na príznakový register (FREG)

Príznaky sú iba čítané.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Základný stavebný prvok viacnásobnej presnej aritmetiky.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

JNZ

Podmienený skok pri nenulovom stave

01011
Skok

Mnemonic	JNZ
Opcode	01011 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Skok
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Podmienený skok pri nenulovom stave.

Architektonická sémantika

```
Signatúra:  JNZ target(IMM|REG) : if FREG.ZF == 0 then PC = target
Operácia:  if FREG.ZF == 0 then PC <- target
Príklad:  JNZ R3 ; Skok na adresu R3, ak ZF == 0 (nie je nula)
```

Konfigurácia a operandy



Oktet 0: kód operácie 01011, bity $[2:0]$ (MOD): 000 (16-bitová adresa Imm, 3 bajty), 001 (nepriamo cez Rs, 2 bajty), 010 (24-bitový cez PTREG). Skok sa vykoná pri nulovom stave príznaku FREG.ZF = 0.

Vplyv na príznakový register (FREG)

Príznaky zostávajú nezmenené.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Rýchle vyhodnotenie podmienky v mikroprograme.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

JZ

Podmienený skok pri nulovom stave

01100
Skok

Mnemonic	JZ
Opcode	01100 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Skok
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Podmienený skok pri nulovom stave.

Architektonická sémantika

```
Signatúra:  JZ target(IMM|REG) : if FREG.ZF == 1 then PC = target
Operácia:  if FREG.ZF == 1 then PC <- target
Príklad:  JZ R4 ; Skok na adresu R4, ak ZF == 1 (je nula)
```

Konfigurácia a operandy



Oktet 0: kód operácie 01100, bity $[2:0]$ (MOD): 000 (16-bitová adresa Imm, 3 bajty), 001 (nepriamo cez Rs, 2 bajty), 010 (24-bitový cez PTREG). Skok sa vykoná pri nastavenom príznaku nuly FREG . ZF = 1.

Vplyv na príznakový register (FREG)

Príznaky zostávajú nezmenené.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Optimalizované pre reťazcové a vyhľadávacie operácie.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

AND

Bitový logický súčin (AND)

01101
Logika

Mnemonic	AND
Opcode	01101 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Logika
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

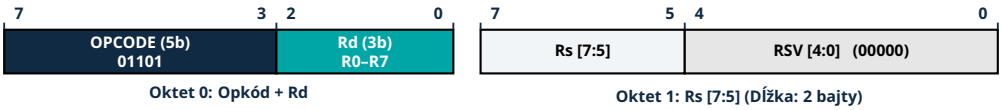
Účel inštrukcie

Bitový logický súčin (AND).

Architektonická sémantika

```
Signatúra:  AND arg0(REG), arg1(REG) : arg0 = arg0 & arg1
Operácia:  arg0 <- arg0 AND arg1
Príklad:  AND R0, R1 ; R0 = R0 & R1
```

Konfigurácia a operandy



Oktet 0: kód operácie 01101, bity $[2:0]$ priamo určujú cieľový register Rd (R0–R7, prvý argument, Rd $\leftarrow$ Rd $\&$ Rs).
Oktet 1: bity $[7:5]$ určujú druhý operand Rs (R0–R7); bity $[4:0]$ obsahujú 00000_2. Dĺžka inštrukcie: 2 bajty.

Vplyv na príznakový register (FREG)

Nastavuje príznak nuly ZF; pretečenie vynulované.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.

4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Kľúčová inštrukcia pre bitové maskovanie.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

OR

Bitový logický súčet (OR)

01110
Logika

Mnemonic	OR
Opcode	01110 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Logika
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

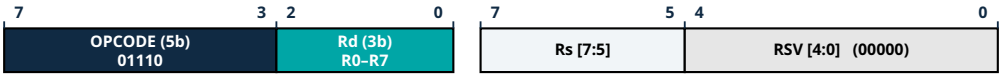
Účel inštrukcie

Bitový logický súčet (OR).

Architektonická sémantika

```
Signatúra:  OR arg0(REG), arg1(REG) : arg0 = arg0 | arg1
Operácia:  arg0 <- arg0 OR arg1
Príklad:  OR R0, R1 ; R0 = R0 | R1
```

Konfigurácia a operandy



Oktet 0: Opkód + Rd

Oktet 1: Rs [7:5] (Dĺžka: 2 bajty)

Oktet 0: kód operácie 01110, bity $[2:0]$ určujú cieľový register Rd (R0–R7, prvý argument, Rd ← Rd | Rs).
Oktet 1: bity $[7:5]$ určujú druhý operand Rs (R0–R7); bity $[4:0]$ obsahujú 00000_2. Dĺžka inštrukcie: 2 bajty.

Vplyv na príznakový register (FREG)

Nastavuje ZF; prenosy sú deterministicky vynulované.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.

4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Rýchle zlučovanie riadiacich príznakov.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

NAND

Bitový logický zápor súčinu (NAND)

01111
Logika

Mnemonic	NAND
Opcode	01111 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Logika
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

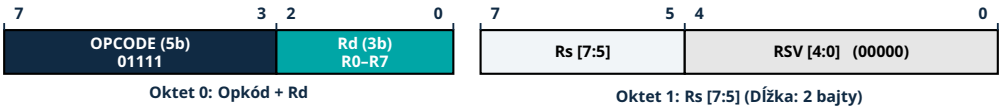
Účel inštrukcie

Bitový logický zápor súčinu (NAND).

Architektonická sémantika

```
Signatúra:  NAND arg0(REG), arg1(REG) : arg0 = ~(arg0 & arg1)
Operácia:  arg0 <- NOT (arg0 AND arg1)
Príklad:  NAND R0, R1 ; R0 = ~(R0 & R1)
```

Konfigurácia a operandy



Oktet 0: kód operácie 01111, bity $[2:0]$ určujú cieľový register Rd (R0–R7, prvý argument).
Oktet 1: bity $[7:5]$ určujú operand Rs (R0–R7); bity $[4:0]$ obsahujú 00000_2. Dĺžka: 2 bajty.

Vplyv na príznakový register (FREG)

Nastavuje príznak ZF.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Umožňuje kompaktnú realizáciu zákazníckych logických funkcií.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

NOR

Bitový logický zápor súčtu (NOR)

10000
Logika

Mnemonic	NOR
Opcode	10000 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Logika
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

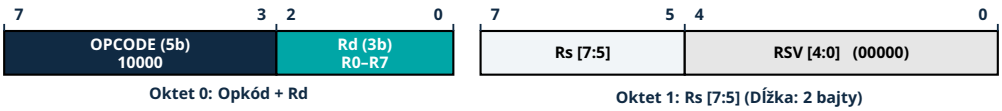
Účel inštrukcie

Bitový logický zápor súčtu (NOR).

Architektonická sémantika

```
Signatúra:  NOR arg0(REG), arg1(REG) : arg0 = ~(arg0 | arg1)
Operácia:  arg0 <- NOT (arg0 OR arg1)
Príklad:  NOR R0, R1 ; R0 = ~(R0 | R1)
```

Konfigurácia a operandy



Oktet 0: kód operácie 10000, bity $[2:0]$ určujú cieľový register Rd (R0–R7, prvý argument).
Oktet 1: bity $[7:5]$ určujú operand Rs (R0–R7); bity $[4:0]$ obsahujú 00000_2. Dĺžka: 2 bajty.

Vplyv na príznakový register (FREG)

Nastavuje príznak ZF.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Rozširuje množinu rýchlych bitových transformácií.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

NOT

Bitová inverzia (jednotkový doplnok)

10001
Logika

Mnemonic	NOT
Opcode	10001 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Logika
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Bitová inverzia (jednotkový doplnok).

Architektonická sémantika

```
Signatúra:  NOT arg0(REG) : arg0 = ~arg0
Operácia:  arg0 <- NOT arg0
Príklad:  NOT R0 ; R0 = ~R0 (1-bajtová inštrukcia)
```

Konfigurácia a operandy



Jednobajtová inštrukcia (dĺžka: 1 bajt). Oktet 0: bity $[7:3]$ obsahujú opkód 10001, bity $[2:0]$ priamo určujú cieľový register Rd (R0–R7). Druhý bajt zo zbernice pamäte sa nenačítava.

Vplyv na príznakový register (FREG)

Nastavuje príznak ZF.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Vykoná sa v minimálnom počte mikrotaktov bez načítania druhého bajtu.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

XOR

Bitová neverzia (XOR)

10010
Logika

Mnemonic	XOR
Opcode	10010 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Logika
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Bitová neverzia (XOR).

Architektonická sémantika

```
Signatúra:  XOR arg0(REG), arg1(REG) : arg0 = arg0 ^ arg1
Operácia:  arg0 <- arg0 XOR arg1
Príklad:  XOR R0, R1 ; R0 = R0 ^ R1 (pri R0==R1 vynuluje R0)
```

Konfigurácia a operandy



Oktet 0: Opkód + Rd

Oktet 1: Rs [7:5] (Dĺžka: 2 bajty)

Oktet 0: kód operácie 10010, bity $[2:0]$ určujú cieľový register Rd (R0–R7, prvý argument, Rd $\leftarrow$ Rd $\oplus$ Rs).
Oktet 1: bity $[7:5]$ určujú druhý operand Rs (R0–R7); bity $[4:0]$ obsahujú 00000_2. Dĺžka: 2 bajty.

Vplyv na príznakový register (FREG)

Nastavuje príznak ZF.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Základná inštrukcia pre paritu, CRC a kryptografiu.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

XNOR

Bitová ekvivalencia

10011
Logika

Mnemonic	XNOR
Opcode	10011 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Logika
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

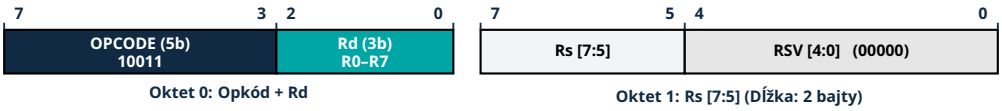
Účel inštrukcie

Bitová ekvivalencia.

Architektonická sémantika

```
Signatúra:  XNOR arg0(REG), arg1(REG) : arg0 = ~(arg0 ^ arg1)
Operácia:  arg0 <- NOT (arg0 XOR arg1)
Príklad:  XNOR R0, R1 ; R0 = ~(R0 ^ R1)
```

Konfigurácia a operandy



Oktet 0: kód operácie 10011, bity $[2:0]$ určujú cieľový register Rd (R0–R7, prvý argument).
Oktet 1: bity $[7:5]$ určujú operand Rs (R0–R7); bity $[4:0]$ obsahujú 00000_2. Dĺžka: 2 bajty.

Vplyv na príznakový register (FREG)

Nastavuje príznak ZF.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Rýchla paralelná kontrola ekvivalencie masiek.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

INC

Inkrementácia o 1

10100
Aritmetika

Mnemonic	INC
Opcode	10100 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Aritmetika
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Inkrementácia o 1.

Architektonická sémantika

```
Signatúra:  INC arg0(REG) : arg0 = arg0 + 1
Operácia:  arg0 <- arg0 + 1
Príklad:  INC R0 ; R0 = R0 + 1 (1-bajtová inštrukcia)
```

Konfigurácia a operandy



Oktet 0 (1 bajt, 0 doplň. bajtov)

Jednobajtová inštrukcia (dĺžka: 1 bajt). Oktet 0: bity $[7:3]$ obsahujú opkód 10100, bity $[2:0]$ priamo určujú register Rd (R0–R7). Druhý bajt zo zbernice pamäte sa nenačítava.

Vplyv na príznakový register (FREG)

Nastavuje ZF. Pretečenie je monitorované v ALU.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Rýchly krok počítadla cyklu vykonaný v minimálnom počte taktov bez ďalšieho čítania.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

DEC

Dekrementácia o 1

10101
Aritmetika

Mnemonic	DEC
Opcode	10101 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Aritmetika
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Dekrementácia o 1.

Architektonická sémantika

```
Signatúra:  DEC arg0(REG) : arg0 = arg0 - 1
Operácia:  arg0 <- arg0 - 1
Príklad:  DEC R0 ; R0 = R0 - 1 (1-bajtová inštrukcia)
```

Konfigurácia a operandy



Jednobajtová inštrukcia (dĺžka: 1 bajt). Oktet 0: bity $[7:3]$ obsahujú opkód 10101, bity $[2:0]$ priamo určujú register Rd (R0–R7). Druhý bajt zo zbernice pamäte sa nenačítava.

Vplyv na príznakový register (FREG)

Nastavuje ZF. Výpožička je monitorovaná v ALU.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Kompaktná realizácia cyklov bez načítania druhého bajtu.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

PUSH

Uloženie slova do zásobníka

10110
Zásobník

Mnemonic	PUSH
Opcode	10110 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Zásobník
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

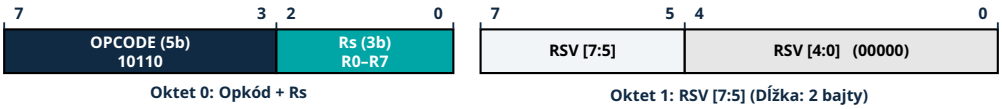
Účel inštrukcie

Uloženie slova do zásobníka.

Architektonická sémantika

```
Signatúra:  PUSH arg0(REG) : SREG -= 2, MEM[SREG] = arg0
Operácia:  SREG <- SREG - 2, MEM[SREG] <- arg0
Príklad:  PUSH R0 ; Uložiť slovo R0 na vrchol zásobníka
```

Konfigurácia a operandy



Oktet 0: kód operácie 10110, bity $[2:0]$ priamo určujú ukladaný zdrojový register Rs (R0–R7, prvý argument).
Oktet 1: bity $[7:0]$ obsahujú 00000000_2 (rezerva). Dĺžka inštrukcie: 2 bajty.

Vplyv na príznakový register (FREG)

Aritmetické príznaky sa nemenia.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Hardvérová podpora volaní funkcií a ukladania kontextu.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

POP

Výber slova zo zásobníka

10111
Zásobník

Mnemonic	POP
Opcode	10111 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Zásobník
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

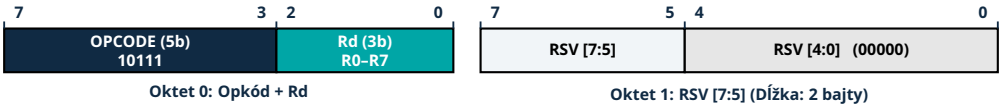
Účel inštrukcie

Výber slova zo zásobníka.

Architektonická sémantika

```
Signatúra:  POP arg0(REG) : arg0 = MEM[SREG], SREG += 2
Operácia:  arg0 <- MEM[SREG], SREG <- SREG + 2
Príklad:  POP R0 ; Vybrať slovo z vrcholu zásobníka do R0
```

Konfigurácia a operandy



Oktet 0: kód operácie 10111, bity $[2:0]$ priamo určujú cieľový register-príjemcu Rd (R0–R7, prvý argument).
Oktet 1: bity $[7:0]$ obsahujú 00000000_2 (rezerva). Dĺžka inštrukcie: 2 bajty.

Vplyv na príznakový register (FREG)

Aritmetické príznaky sa nemenia.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Symetrické obnovenie kontextu po prerušení.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

BSL

Hardvérový posun doľava (iteratívny register)

11000
Posuny

Mnemonic	BSL
Opcode	11000 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Posuny
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Hardvérový posun doľava (iteratívny register).

Architektonická sémantika

```
Signatúra:  BSL arg0(REG), arg1(IMM) : arg0 = arg0 « arg1
Kódovanie:  Oktet 0:  [7:3]=11000, [2:0]=Rd (cieľový register R0–R7)
            Oktet 1:  [7:0]=shift (počet bitov posunu 0–16)
Operácia:  Postupný taktovaný posun registra Rd doľava o shift bitov:
Hardvérový cyklus (1 takt na 1 bit, ≈ 300 tranzistorov):
Pre každý takt t ∈ [1..shift]:
Najvyšší bit 15 sa vytlačí (last_displaced <- bit 15);
Pri SMOD = 1 sa do najnižšieho bitu 0 vráti vytlačený bit (rotácia);
Pri SMOD = 0 sa do najnižšieho bitu 0 vloží bit z FREG.DBIT (0 alebo 1);
Zachytenie:  FREG.DBIT <- last_displaced (posledný vytlačený bit).
Príklad:  BSL R0, 4 ; Posun R0 doľava o 4 bity (trvá 4 takty posunu)
```

Konfigurácia a operandy



Oktet 0: kód operácie 11000, bity $[2:0]$ priamo určujú posúvaný register Rd (R0–R7).
Oktet 1: celý bajt ([7:0]) určuje počet taktov posunu shift (0–16). Fyzicky sa posun realizuje taktovaným cyklom na 16 D-klopných obvodoch. Dĺžka inštrukcie: 2 bajty.

Vplyv na príznakový register (FREG)

Posledný vytlačený bit sa uloží do FREG.DBIT (bit 13). Príznak nuly FREG.ZF sa nastaví pri nulovom výsledku. Riadené príznakmi FREG.SMOD (bit 14) a FREG.DBIT (bit 13).

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Iteratívny posuvný register na D-klopných obvodoch: cena len ≈ 300 tranzistorov namiesto $\approx 30\,000$ pri jedno-taktovom maticovom posúvači. Posun o 16 bitov vyžaduje o 15 taktov viac než posun o 1 bit.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

BSR

Hardvérový posun doprava (iteratívny register)

11001
Posuny

Mnemonic	BSR
Opcode	11001 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Posuny
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Hardvérový posun doprava (iteratívny register).

Architektonická sémantika

```
Signatúra:  BSR arg0(REG), arg1(IMM) : arg0 = arg0 » arg1
Kódovanie:  Oktet 0:  [7:3]=11001, [2:0]=Rd (cieľový register R0–R7)
            Oktet 1:  [7:0]=shift (počet bitov posunu 0–16)
Operácia:  Postupný taktovaný posun registra Rd doprava o shift bitov:
Hardvérový cyklus (1 takt na 1 bit, ≈ 300 tranzistorov):
Pre každý takt t ∈ [1..shift]:
Najnižší bit 0 sa vytlačí (last_displaced <- bit 0);
Pri SMOD = 1 sa do najvyššieho bitu 15 vráti vytlačený bit (rotácia);
Pri SMOD = 0 sa do najvyššieho bitu 15 vloží bit z FREG.DBIT (0 alebo 1);
Zachytenie:  FREG.DBIT <- last_displaced (posledný vytlačený bit).
Príklad:  BSR R0, 4 ; Posun R0 doprava o 4 bity (trvá 4 takty posunu)
```

Konfigurácia a operandy



Oktet 0: kód operácie 11001, bity $[2:0]$ priamo určujú posúvaný register Rd (R0–R7).
Oktet 1: celý bajt ([7:0]) určuje počet taktov posunu shift (0–16). Fyzicky sa posun realizuje taktovaným cyklom na 16 D-klopných obvodoch. Dĺžka inštrukcie: 2 bajty.

Vplyv na príznakový register (FREG)

Posledný vytlačený bit sa uloží do FREG.DBIT (bit 13). Príznak nuly FREG.ZF sa nastaví pri nulovom výsledku. Riadené príznakmi FREG.SMOD (bit 14) a FREG.DBIT (bit 13).

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Iteratívny posuvný register na D-klopných obvodoch: cena len ≈ 300 tranzistorov namiesto $\approx 30\,000$ pri jedno-taktovom maticovom posúvači. Posun o 16 bitov vyžaduje o 15 taktov viac než posun o 1 bit.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

MSP

Manipulácia so špeciálnymi registrami (FREG, PTREG, SREG, VEC)

11010
Systém

Mnemonic	MSP
Opcode	11010 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Systém
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Manipulácia so špeciálnymi registrami (FREG, PTREG, SREG, VEC).

Architektonická sémantika

```
Signatúra:  MSP sreg, [op], args :  práca so špeciálnymi registrami (2–6
bajtov)
Operácia:  Priamy zápis (bit 10 = 0):
sreg=00 (FREG): FREG <- Rs[15:0] alebo imm16 / [addr16] / [$ptreg]
sreg=01 (PTREG): 32-bitový ukazovateľ PTREG (4 GB): z GPR, imm32, [$ptreg],
[addr32]
sreg=10 (SREG): 32-bitový ukazovateľ zásobníka SREG: z GPR, imm32, [$ptreg],
[addr32]
sreg=11 (VEC): 32-bitový vektor prerušenia ($ir0..$ir15) alebo zóny
($pv0..$pv15)
Modifikácia (bit 10 = 1):
sreg=00 (FREG): FREG <- FREG ⊙ Mask {AND, OR, XOR, XNOR}
sreg=01 (PTREG): PTREG <- PTREG ± Offset {ADD, SUB}
sreg=10 (SREG): SREG <- SREG ± Offset {ADD, SUB}
Príklad:  MSP.ld $ir0, 0x00010000 ; Nastavenie vektora IRQ0 (6 bajtov)
MSP.ld $pv0, 0x00008000 ; Začiatok chránenej Zóny Privilégií 0
MSP.ld $pv1, 0x00008FFF ; Koniec chránenej Zóny Privilégií 0
MSP.ld $ptreg, [$ptreg] ; Natívne 1-taktové dereferencovanie (2 bajty)
MSP.or $freg, 0x0200 ; Povolenie globálnych prerušení MIE
```

Konfigurácia a operandy



Oktet 0: kód operácie `11010`. Bit $[10]$ (bit 2 oktetu 0) určuje režim: 0 — priame načítanie, 1 — modifikácia. Bity $[9:8]$ (bity 1–0 oktetu 0) určujú selektor: 00 = FREG, 01 = PTREG, 10 = SREG.
Oktet 1: pri načítaní ($[10] = 0$) určuje zdroj: `Rs` pre FREG alebo dvojicu `Rs_hi`, `Rs_lo` pre PTREG/SREG. Pri modifikácii ($[10] = 1$) pre FREG bity $[7:6]$ kódujú operáciu (AND, OR, XOR, XNOR) a bity $[5:3]$ určujú register masky `Rs`; pre PTREG a SREG bit $[7]$ určuje operáciu (0=ADD, 1=SUB) a bity $[6:4]$ určujú register posunu `Rs`. Dĺžka inštrukcie: 2 bajty.

Vplyv na príznakový register (FREG)

Príznaky FREG sa modifikujú priamo alebo maskou. Aritmetika PTREG a SREG generuje príznak prenosu CF cez ALU.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Univerzálny privilegovaný riadiaci primitív systémovej úrovne.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

MOD

Celočíselný zvyšok po delení (modulo)

11011
Aritmetika

Mnemonic	MOD
Opcode	11011 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Aritmetika
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

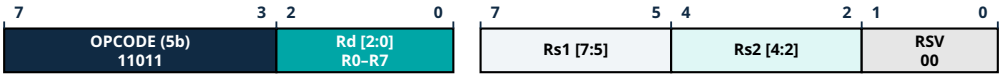
Účel inštrukcie

Celočíselný zvyšok po delení (modulo).

Architektonická sémantika

```
Signatúra:  MOD arg0, arg1, arg2 :  arg0 = arg1 % arg2
Operácia:  dst <- arg1 % arg2
Príklad:  MOD R0, R1, R2 ; R0 = R1 % R2 (zvyšok po delení)
```

Konfigurácia a operandy



Oktet 0: Opkód + Rd (1. arg)

Oktet 1: Rs1 [7:5], Rs2 [4:2], 00 [1:0] (2 bajty)

Oktet 0: kód operácie 11011, bity $[2:0]$ zadávajú cieľový register Rd (R0–R7, prvý argument, príjemca zvyšku).
Oktet 1: bity $[7:5]$ určujú delenec Rs1 (R0–R7); bity $[4:2]$ určujú deliteľ Rs2 (R0–R7); bity $[1:0]$ obsahujú 00. Dĺžka inštrukcie: 2 bajty.

Vplyv na príznakový register (FREG)

Pri delení nulou sa aktivuje hardvérový chybový príznak EM (Error Math) bez zamrznutia jadra; aktualizuje ZF.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).

3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Vypočíta zvyšok po celočíselnom delení delenca Rs1 deliteľom Rs2 a uloží výsledok priamo do cieľového registra Rd bez prepisovania medzivýsledkov v akumulátoroch.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

INTR

Softvérové prerušenie podľa tabuľky vektorov (2 bajty)

11100
Systém

Mnemonic	INTR
Opcode	11100 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Systém
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Softvérové prerušenie podľa tabuľky vektorov (2 bajty).

Architektonická sémantika

```
Signatúra:  INTR vec(IMM4) / INT vec :  vyvolanie vektora prerušenia 0..15
Operácia:  SREG <- SREG - 4; MEM32[SREG] <- PC
SREG <- SREG - 2; MEM16[SREG] <- FREG
FREG.INTR <- 1; FREG.PRV <- 1; FREG.MIE <- 0; PC <- VECTORS[vec]
Príklad:  INT 5 ; Softvérové prerušenie podľa vektora č.  5 (2 bajty)
```

Konfigurácia a operandy



Oktet 0: kód operácie 11100, najnižšie 3 bity $[2:0]$ obsahujú 000. **Oktet 1:** najvyššie 4 bity $[7:4]$ sú rezervované (0000), zatiaľ čo najnižšie 4 bity $[3:0]$ určujú index vektora vec (0–15) zo 16 systémových vektorov prerušení ($ir0..$ir15). Dĺžka inštrukcie: 2 bajty.

Vplyv na príznakový register (FREG)

Nastaví príznak aktívneho prerušenia FREG.INTR (bit 15) na 1, prejde do režimu supervízora (FREG.PRV = 1) a vynuluje FREG.MIE (bit 9) na 0. 32-bitová návratová adresa a 16-bitový stav FREG sa uložia na zásobník SREG.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Bezpečné systémové volanie (Syscall) alebo ladiaca pasca podľa čísla vektora 0..15 s úplným zachovaním kontextu na zásobníku SREG.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

CALL

Volanie podprogramu s uložením PC na zásobník

11101
Skoky

Mnemonic	CALL
Opcode	11101 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Skoky
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Volanie podprogramu s uložením PC na zásobník.

Architektonická sémantika

```
Signatúra:  CALL target :  SREG -= 4, MEM32[SREG] = PC, PC = target
Kódovanie:  Oktet 0:  [7:3]=11101, [2:0]=cfg (0:  imm16, 1:  Rd, 2:  [$ptreg],
3:  [addr16])
Operácia:  ret_addr <- PC
SREG <- SREG - 4, MEM32[SREG] <- ret_addr
PC <- target
Príklad:  CALL func ; Volanie podprogramu na adrese func (3 bajty)
CALL $0 ; Nepriame volanie cez register R0 (2 bajty)
CALL [$ptreg] ; Nepriame volanie cez 32-bitový ukazovateľ PTREG (2 bajty)
```

Konfigurácia a operandy



Oktet 0: kód operácie 11101, bity $[2:0]$ určujú režim adresácie cieľa skoku (cfg): `0` — priama 16-bitová adresa (3 bajty), 1 — nepriamo cez register Rd (2 bajty), 2 — cez 32-bitový ukazovateľ PTREG (2 bajty), 3 — nepriamo cez pamäťový ukazovateľ (3 bajty).
Pred vykonaním skoku sa celá 32-bitová návratová adresa (adresa nasledujúcej inštrukcie) zapíše na vrchol systémového zásobníka SREG.

Vplyv na príznakový register (FREG)

Príznaky sa nemenia.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Štandardná inštrukcia volania procedúr a funkcií so zachovaním 32-bitového kontextu na zásobníku SREG.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

RET

Návrat z podprogramu s obnovením PC zo zásobníka

11110
Skoky

Mnemonic	RET
Opcode	11110 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Skoky
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Návrat z podprogramu s obnovením PC zo zásobníka.

Architektonická sémantika

```
Signatúra:  RET : PC = MEM32[SREG], SREG += 4
Operácia:  PC <- MEM32[SREG], SREG <- SREG + 4
Príklad:  RET ; Návrat z podprogramu (1 bajt)
```

Konfigurácia a operandy



Jednobajtová inštrukcia (dĺžka: 1 bajt). Oktet 0: kód operácie 11110, bity $[2:0]$ obsahujú 000. Obnoví 32-bitový programový čítač PC z vrcholu systémového zásobníka SREG a uvoľní 4 bajty zásobníka (SREG += 4).

Vplyv na príznakový register (FREG)

Príznaky sa nemenia.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

Poznámky k implementácii

Symetrická inštrukcia návratu z procedúr a funkcií volaných inštrukciou CALL.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

HALT

Hardvérové zastavenie procesora

11111
Riadenie

Mnemonic	HALT
Opcode	11111 (Horných 5 bitov inštrukčného slova)
Formát inštrukcie	5-bitový opcode, 3-bitové pole cfg a dynamické argumenty (pri špecializovaných príkazoch argumenty priamo za opkódom)
Trieda operácie	Riadenie
Stav ratifikácie	Základná špecifikácia schválená; rozvrhnutie bitových polí argumentov vo fáze finalizácie

Účel inštrukcie

Hardvérové zastavenie procesora.

Architektonická sémantika

```
Signatúra:  HALT / HLT : zastavenie jadra
Operácia:  CPU.HALTED <- 1, PINS.HALT <- 1
Príklad:  HALT ; Úplné zastavenie procesora (1 bajt)
```

Konfigurácia a operandy



Oktet 0 (1 bajt, 0 doplň. bajtov)

Jednobajtová inštrukcia (dĺžka: 1 bajt). Oktet 0: kód operácie 11111, bity $[2:0]$ obsahujú `000`. Prepne procesor do stavu zastavenia jadra až do prijatia hardvérového signálu RESET alebo prerušenia IRQ.

Vplyv na príznakový register (FREG)

Príznaky sa nemenia.

Mikrooperácie dátovej cesty

1. Multiplexorový výber zdrojových operandov z registrového poľa alebo priamej zbernice.
2. Odoslanie operácie do vyhradenej funkčnej jednotky (ALU / pamäť / riadenie).
3. Smerovanie výsledku cez selektor spätného zápisu (MUX 2) do cieľového registra.
4. Synchrónna aktualizácia stavových príznakov FREG a registrových ukazovateľov (PTREG, SREG).

Presné cyklické časovanie a riadiace impulzy: **TBD**.

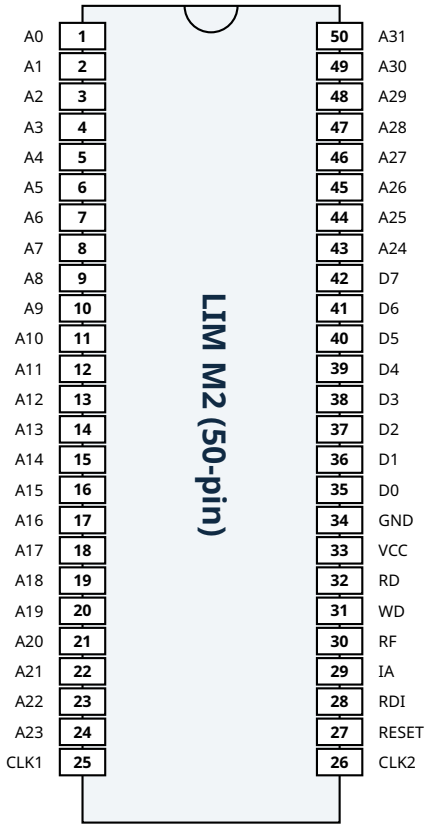
Poznámky k implementácii

Ukončenie behu programu alebo prechod do úsporného režimu.

Polia s označením TBD špecifikujú konfigurovateľné hardvérové voľby.

7 Fyzické rozhranie a rozloženie vývodov (50-vývodové puzdro)

Procesor je integrovaný vo vysokospoľahlivom 50-vývodovom puzdre DIP-50 / QFP-52 s natívnou 32-bitovou adresnou zbernicou a 8-bitovou (jednobajtovou) vonkajšou dátovou zbernicou:



7.1 Funkčný popis signálov

Vývod	Typ	Popis systémového signálu
A0–A31	Výstup / Tri-State	32-bitová natívna adresová zbernica (priamy fyzický prístup k 4 GB pamäte bez okien a bánk).
D0–D7	Obojsmerný	8-bitová (jednobajtová) vonkajšia zbernica pre prenos dát s pamäťou a perifériami.
CLK1, CLK2	Vstup	Dvojfázové hodinové signály s ochranou proti prekrytiu fáz.
RESET	Vstup	Hardvérový resetovací signál procesora (aktívna nula).
RDI	Vstup	Raw Direct Interrupt: prioritná linka hardvérového prerušenia.
IA	Výstup	Interruption Accepted / Interrupt Acknowledge: potvrdenie prijatia prerušenia perifériám.
RF	Vstup	Ready Flag: hardvérové potvrdenie pripravenosti podriadeného zariadenia pamäte/periférie.
RD, WD	Výstup	Riadiace impulzy čítania (Read Enable) a zápisu (Write Enable) systémovej zbernice.
VCC, GND	Napájanie	Hlavné digitálne napájanie logiky procesora (+3.3 V / +5 V nominálne) a zem.

8 Podsystém prerušení a vektorové spracovanie

Podsystém prerušení procesora LIM M2 je navrhnutý tak, aby poskytoval minimálnu latenciu odozvy na udalosti v reálnom čase, prísny determinizmus a úplnú ochranu pred zbernicovými kolíziami. Architektúra zjednocuje externé hardvérové požiadavky a interné softvérové pasce do jednotnej 16-vektorovej hierarchie s hardvérovým potvrdzovaním na fyzickej úrovni.

8.1 Zjednotený priestor 16 vektorov prerušení

Procesor podporuje presne 16 systémových vektorov (indexy 0–15), ktoré sú ortogonálne zdieľané medzi externými perifériami a internými udalosťami procesorového jadra:

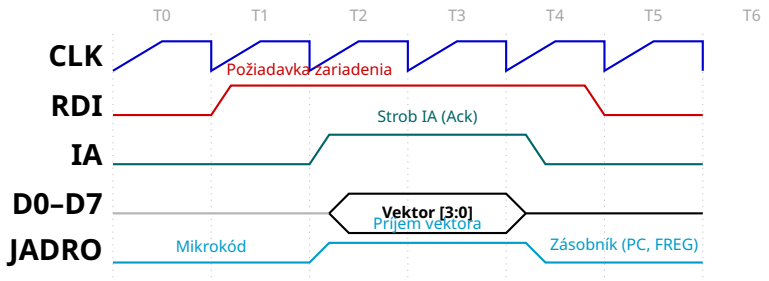
Table 3: Zjednotená tabuľka 16 vektorov prerušení LIM M2

Vektor	Kód	Zdroj / Typ	Funkčné určenie
VEC 0	`0x00`	Hardvérový / Reset	Power-on Reset: počiatočná inicializácia jadra a registrov.
VEC 1	`0x01`	Hardvérový (NMI)	Nemaskovateľné prerušenie: výpadok napájania, kritická porucha.
VEC 2	`0x02`	Softvérová pasca	Aritmetická výnimka (delenie nulou, príznak `FREG.EM`).
VEC 3	`0x03`	Softvérová pasca	Systémové volanie jadra (Syscall / API dispečer OS).
VEC 4	`0x04`	Hardvérový / Perif.	Systémový intervalový časovač / Kvantovač plánovača úloh.
VEC 5	`0x05`	Hardvérový / Perif.	Sériové komunikačné rozhranie (UART RX/TX).
VEC 6	`0x06`	Hardvérový / Perif.	Radič priameho prístupu do pamäte (DMA Transfer Complete).
VEC 7	`0x07`	Hardvérový / Perif.	Externý radič prerušení / Zbernicový most.
VEC 8–15	`0x08–0x0F`	Zdieľané (HW / SW)	Používateľské hardvérové linky a softvérové vektory.

8.2 Hardvérový protokol potvrdzovania (RDI → IA → D0–D7)

Pre spoľahlivú komunikáciu s externými radičmi je implementovaný 4-fázový cyklus hardvérového potvrdzovania (handshake), ktorý úplne eliminuje réžiu cyklického dopytovania (polling):

1. **Fáza požiadavky (RDI):** Periférne zariadenie nastaví aktívnu vysokú úroveň na vstupe `RDI` (*Raw Direct Interrupt*). Linka je testovaná jadrom na atomických hraniciach mikroinštrukcií.
2. **Fáza kontroly masky (Arbitrácia):** Ak je globálne maskovanie prerušení povolené (bit `FREG.MIE` = 1), procesor požiadavku zachytí a dokončí aktuálnu mikrooperáciu dátovej cesty.
3. **Fáza potvrdenia (Strob IA):** Jadro vygeneruje aktívny strob na výstupe `IA` (*Interruption Accepted / Interrupt Acknowledge*, pin 26). Tento strob informuje všetky periférie na zbernici o pripravenosti jadra prijať číslo vektora.
4. **Fáza prenosu vektora (Zbernica D0–D7):** Súčasne s aktiváciou `IA` sa dátová zbernica `D0–D7` prepne do režimu vstupu a aktivuje sa signál čítania `RD`. Žiadajúce zariadenie vystaví na linky `D0–D7` 8-bitový identifikátor vektora (spodné 4 bity adresujú vektory 0–15).
5. **Fáza záchytu a prechod:** Procesor zachytí dáta zo zbernice `D0–D7`, deaktivuje signál `IA` a prejde k uloženiu kontextu.



8.3 Uloženie kontextu v 24-bitovom zásobníku a vstup do obsluhy

Prijatie prerušenia je povolené výlučne za podmienky, že procesor už nevykonáva iné prerušenie (FREG.INTR = 0) a prerušenia sú globálne povolené (FREG.MIE = 1). Ak FREG.INTR = 1, akékoľvek ďalšie hardvérové prerušenia sú hardvérovo zablokované.

Po potvrdení prerušenia jadro vykoná nasledujúci postup ochrany stavu:

1. **Uloženie čítača inštrukcií (PC):** Aktuálna hodnota PC sa uloží na vrchol systémového zásobníka posunom 24-bitového ukazovateľa SREG:

$$\text{SREG} \leftarrow \text{SREG} - 2, \quad \text{Memory}[\text{SREG}] \leftarrow \text{PC}_{15:0}$$

2. **Uloženie príznakov a stavu (FREG):** Aktuálny stav registra FREG (s pôvodnou hodnotou FREG.INTR=0) sa zapíše do zásobníka:

$$\text{SREG} \leftarrow \text{SREG} - 2, \quad \text{Memory}[\text{SREG}] \leftarrow \text{FREG}$$

3. **Hardvérové blokovanie vnorených prerušení:** Jadro nastaví príznak vykonávania prerušenia FREG.INTR $\leftarrow$ 1 (bit 15) a vynuluje masku FREG.MIE $\leftarrow$ 0, čím sa zamedzí kaskádovým reentrantným volaniam.
4. **Skok na vektor:** Adresa obslužnej rutiny sa vypočíta z prijatého čísla vektora a zapíše do PC.

8.4 Návrat z prerušenia

Ukončenie obsluhy prerušenia obnoví pôvodný vykonávaný kontext zo zásobníka:

1. Zo zásobníka sa načíta a obnoví pôvodný stav registra príznakov:

$$\text{FREG} \leftarrow \text{Memory}[\text{SREG}], \quad \text{SREG} \leftarrow \text{SREG} + 2$$

   (Obnovením registra FREG sa zároveň bit FREG.INTR vráti na 0, čím sa uvoľní blokovanie, a hodnota FREG.MIE sa vráti na pôvodný stav).
2. Zo zásobníka sa obnoví čítač inštrukcií:

$$\text{PC} \leftarrow \text{Memory}[\text{SREG}], \quad \text{SREG} \leftarrow \text{SREG} + 2$$

3. Procesor plynule pokračuje vo vykonávaní prerušeného toku inštrukcií v nasledujúcom takte bez akejkoľvek straty údajov.

9 Jazyk symbolických inštrukcií (Assembler) a kompilátor LIM M2 (`limasm.py`)

Dvojprechodový kompilátor assemblera `LIM2/sim/limasm.py` prekladá zdrojový kód (`.asm` / `.lasm`) súčasne do binárneho obrazu (`.bin`) a hexadecimálneho súboru (`.hex`).

9.1 Pomenovanie registrov a prípony veľkosti operandov

- **Všeobecné registre (16-bitové):** označujú sa prefixom `$`: `$0`–`$7` (alebo R0–R7).
- **Špeciálne systémové registre:** `$ptreg` (24-bitový ukazovateľ PTREG), `$freg` (16-bitový stavový register FREG), `$sreg` / `$sp` (24-bitový ukazovateľ zásobníka SREG), `$vec0`–`$vec7`.
- **Prípony veľkosti (`.b` / `.w` / `.d`):** XXX`.b` (byte, 8 bitov), XXX`.w` (word, 16 bitov, predvolené), XXX`.d` (doubleword, 24/32 bitov).

9.2 Aritmetika, unárne inštrukcie a prístup do pamäte (`LMR`, `LRM`)

```
ADD $0, $1, $2          ; $0 = $1 + $2 (alebo ADD R0, R1, R1; rovnako SUB, MUL, DIV
INC $0                  ; $0 = $0 + 1  (1-bajtová inštrukcia; rovnako DEC, NOT)
LMR.w $0, 0x1234        ; Číslo bez zátvoriek -> priama konštanta (SRC=0)
LMR.w $0, [0x1000]      ; Hranaté zátvorky [addr] -> odkaz na údaje v pamäti (SRC=1
LMR.w $0, [$ptreg]      ; Odkaz cez 24-bitový ukazovateľ PTREG (SRC=2)
LMR $0, [$0:$1]         ; 24-bitový ukazovateľ z dvojice registrov [$0:$1] (SRC=3)
```

9.3 Špeciálne registre (`MSP`) a viacnásobné dereferencovanie cez PTREG

```
MSP.and $freg, 0x0001       ; Maska AND na stavový register: FREG &= 0x0001
MSP.ld  $freg, 0x0000       ; Priame nastavenie stavového registra FREG
MSP.ld  $ptreg, [$0:$1]     ; Zostavenie 32-bitového ukazovateľa do PTREG z dvojice
MSP.ld  $sreg, [$0:$1]      ; Načítanie 32-bitového ukazovateľa zásobníka SREG z dv
MSP.ld  $ptreg, 0xA000      ; Priama adresa 0xA000 do PTREG
MSP.ld  $ptreg, [0xA000]    ; Načítanie ukazovateľa z pamäti [0xA000] do PTREG

; Natívne viacnásobné dereferencovanie reťazca ukazovateľov cez PTREG:
MSP.ld  $ptreg, 0xA000      ; Koreňová adresa ukazovateľa
MSP.ld  $ptreg, [$ptreg]    ; 1. dereferencovanie: PTREG <- MEM[PTREG]
MSP.ld  $ptreg, [$ptreg]    ; 2. dereferencovanie: PTREG <- MEM[PTREG]
MSP.ld  $ptreg, [$ptreg]    ; 3. dereferencovanie: PTREG <- MEM[PTREG]
LMR.w   $0, [$ptreg]        ; Načítanie cieľovej hodnoty do $0
```

Kompilácia do `.bin` a `.hex`: `python3 LIM2/sim/limasm.py LIM2/sim/demo.asm -l`.

10 Záver

Architektúra **LIM M2** spája overenú stabilitu mikroprogramového riadenia s modernou efektivitou aktívnej dátovej cesty bez tretieho stavu. Odstránením zbernicových kolízií a parazitných kapacít jadro zaručuje kryštálovú čistotu digitálnych signálov a bezpečne odomyká frekvenčný rozsah od 200 MHz až po fyzické limity moderných kremíkových štruktúr.